CSC490 Assignment 4: RL-ing Nanochat

Sohee Goo / 1008065479 Akshaya Deepak Ramachandran / 1008806810
Maryam Taj / 1008990362 Kashish Mittal / 1009115315

Contents

NOTE: Our forked repository and code changes were made here: https://github.com/soheegoo/nanochat. We used nanochat_modal.py to run our experiments.

1 Part 1

Karpathy's RL implementation in nanochat is a simpler version of the standard GRPO which fits the context of training a small model on GSM8K with binary rewards.

First, the training is on-policy. Each set of rollouts is used for exactly one gradient update and then discarded. Standard GRPO reuses the same rollouts for multiple gradient steps to save compute, since generating rollouts from large models is expensive. GRPO uses PPO-style importance sampling ratios with clipping to track how much the model's token probabilities have shifted since generation, and capping updates when the data becomes too stale. Karpathy doesn't do this because nanochat is a relatively small model, generating fresh rollouts is cheaper, so there is no need to squeeze multiple updates out of each batch. With fresh data every step, the importance ratio is always 1.0 and clipping has no effect, so both are dropped entirely. The result is a simpler loss function like REINFORCE: just log_prob $\times$ advantage.

Second, there is no KL divergence penalty against a reference model. In standard GRPO, a frozen copy of the original model is kept in memory, and a penalty term discourages the training policy from drifting too far from it. This guards against reward hacking, where the model finds degenerate shortcuts to maximize reward. Karpathy doesn't do this, since it halves the memory requirement by not storing a second model, and the risk of reward hacking is lower here because GSM8K has a strict binary reward with short completions, leaving less room for exploitation. The on-policy setup also provides stability since the model only learns from its own current behavior, not from increasingly distant old data. Another reason could be that nanochat only runs RL for a single epoch over GSM8K. With short training, the model doesn't have enough updates to drift far from its starting point. KL regularization is most important when training runs for a long time and the policy has opportunity to gradually collapse.

Third, advantages are computed as (reward − mean) rather than the z-score (reward − mean) / std. Dividing by standard deviation makes gradient magnitudes consistent across batches regardless of reward spread. However, when all samples for a question get the same reward (e.g., all correct or all wrong), the standard deviation approaches zero, causing numerical instability. Karpathy's formulation is simpler. When all samples score the same, advantages are zero, and the model correctly learns nothing from that question since there is no signal about which responses are better. With only 16 samples per question and a binary reward, the standard deviation estimate is inherently noisy. Dividing by a noisy σ computed from just 16 binary samples can amplify or suppress gradients unpredictably. Skipping the division avoids introducing that noise source.

Fourth, the loss is normalized at the token level rather than the sequence level. Standard GRPO divides by the number of sequences, meaning a 100-token completion generates 10$\times$ more gradient than a 10-token completion with the same advantage. This implicitly rewards verbosity, since the model learns that longer correct answers contribute more to the objective. Karpathy instead divides by the total number of valid tokens across all sequences (following the DAPO approach), so each token contributes equally regardless of which sequence it belongs to. This is important for GSM8K specifically, where answer lengths vary between simple one-step and complex multi-step problems.

Basically Kaparthy reduces GRPO to token-level REINFORCE with a group-mean baseline. The core insight from GRPO, comparing multiple samples of the same question to compute relative advantages is what is used. But everything needed for off-policy stability (clipping, KL penalties, variance normalization) is removed because it is unnecessary for a small model doing single-step on-policy updates with a simple binary reward signal.

2 Part 2

2.1 Pretrained Model vs Model after SFT

NOTE: Please refer to the Appendix for the graphs from WandB.

Pretraining setup. We pretrain the nanochat baseline model with depth 24, sequence length 2048, and a target parameter:data ratio of 10.5, following the original nanochat configuration.

Using an AdamW optimizer with separate learning rates for embeddings, matrices, and scalars, we train on 7.66B tokens for 7,308 iterations, corresponding to approximately 3.8×10^{19} training FLOPs and a measured MFU of 50.6% over 157.6 minutes on 8$\times$ H100 GPUs.

We decided to go with the baseline nanochat without our ablations from A3 as we wanted a reliable baseline for our RL reward experiments.

SFT setup. Starting from the pretrained checkpoint, we run supervised fine-tuning (SFT) for 500 iterations using the original training mixture (SmolTalk-style conversations, identity and spelling tasks, MMLU, ARC, GSM8K, and SpellingBee). We keep the same sequence length and effective batch size (device batch size 8) and use the default SFT learning rate schedule.

Comparing Metrics

Metric	Score
ARC-Easy	0.5699
ARC-Challenge	0.4121
MMLU	0.3710
GSM8K	0.0735
HumanEval	0.0061
SpellingBee	0.9961
ChatCORE	0.3133

Table 1: Chat results for the baseline SFT model

Category	Metric	Score
Training	CORE	0.2410
Training	Train BPB	0.7528
Training	Val BPB	0.7537
Reasoning	HellaSwag (0-shot)	0.3446
Reasoning	HellaSwag	0.3467
Reasoning	ARC-Easy	0.5488
Reasoning	ARC-Challenge	0.1570
Reasoning	COPA	0.2800
Reasoning	CommonsenseQA	0.0940
Reasoning	PIQA	0.4331
Reasoning	OpenBookQA	0.1840
Language	LAMBADA	0.4366
Language	Winograd	0.3187
Language	WinoGrande	0.1097
Knowledge	Jeopardy	0.1885
Knowledge	BB QA Wikidata	0.5015
Knowledge	BoolQ	-0.1637
BigBench	Dyck Languages	0.0940
BigBench	CS Algorithms	0.3780
BigBench	Operators	0.1810
BigBench	Repeat Copy Logic	0.0000
BigBench	Language ID	0.1735
QA	SQuAD	0.3488
QA	CoQA	0.2604
Evaluation	AGI Eval LSAT-AR	0.0870

Table 2: Base model evaluation results (Pretrained Model)

Rather than comparing individual metrics directly, we group evaluations by capability. Across **reasoning tasks**, SFT produces consistent gains, with ARC-Challenge showing the largest improvement ($0.1570 \rightarrow 0.4121$), suggesting instruction tuning sharpens the model's ability to select and justify answers. ARC-Easy also improves slightly ($0.5488 \rightarrow 0.5699$).

Several **instruction-following benchmarks** (MMLU, GSM8K, HumanEval, SpellingBee) are only evaluated post-SFT, as they require structured prompts and formatted outputs, capabilities learned during fine-tuning rather than pretraining. The model achieves 0.3710 on MMLU and 0.0735 on GSM8K. SpellingBee (0.9961) and ChatCORE (0.3133) confirm the model has acquired reliable instruction-following behavior.

Overall, pretraining equips the model with general linguistic and reasoning priors, while SFT aligns it to instruction-following.

2.2 Updating the SFT step with additional datasets

2.2.1 Dataset selection and justification

Databricks Dolly 15k [6] We include Databricks Dolly 15k to improve the model's general conversational and instruction-following ability. Dolly contains human-curated instructions and responses across diverse tasks, including creative writing, closed and open QA, summarization, information extraction, classification, and brainstorming, primarily sourced from human annotators and Wikipedia. This breadth helps the model handle a wide range of natural language tasks and maintain chat quality while we push harder on math. The main downside is that many tasks are not directly math-related, so some SFT capacity is spent on abilities that do not directly translate into GSM8K performance.

Orca Math Word Problems [7] We add `microsoft/orca-math-word-problems-200k` to provide high-quality grade-school word problems that closely resemble GSM8K. This dataset contains roughly 200k math word problems with step-by-step, GPT-4-Turbo-generated solutions, explicitly designed to train small LMs for multi-step numerical reasoning on GSM8K-style tasks. Its strong distributional alignment and chain-of-thought style make it a natural choice for improving math reasoning. However, it is fully synthetic, so it may contain noisy or subtly incorrect solutions, and over-reliance on its style could reduce robustness to more natural human-written problems.

NuminaMath-CoT [8] We use `AI-MO/NuminaMath-CoT` to expose the model to more challenging math reasoning. NuminaMath-CoT provides roughly 860k problems with chain-of-thought solutions, spanning Chinese high school exercises to international math olympiad problems, all formatted as explicit multi-step reasoning. This gives the model practice with deeper, longer reasoning chains than GSM8K, which we hope will transfer to easier grade-school problems. The trade-off is that the difficulty distribution is significantly harder than GSM8K and largely synthetic, so some examples may be out-of-distribution for our small model and could introduce instability or failure to converge on simpler formats.

whynlp/gsm8k-aug [5] We include `whynlp/gsm8k-aug` as a task-aligned augmentation of our target benchmark. This dataset expands the original GSM8K training set to 385k examples by prompting GPT-4 to generate additional problems and chain-of-thought solutions. Because it closely matches GSM8K in both content and format, it directly targets our objective of improving GSM8K accuracy. The main concern is that, as a GPT-4-augmented synthetic dataset, it may encode particular stylistic biases or repeated patterns that do not appear in the original GSM8K test set.

Data contamination and experimental design To mitigate data contamination, we verified that none of our additional datasets contain GSM8K test split examples. Dolly 15k and gsm8k-aug are derived from independent sources (human annotators and GPT-4 augmentation of the GSM8K training split respectively), and we explicitly exclude the GSM8K test split from all SFT mixtures. For NuminaMath-CoT and Orca-Math, which are synthetically generated, we rely on the dataset authors' contamination analysis confirming no overlap with standard math benchmarks' test sets.

We design two experiments to test distinct hypotheses about how best to incorporate additional data, comparing them against the baseline SFT checkpoint.

We limit each experiment to adding only 2–3 datasets. Prior work shows that (i) general instruction-following gains can plateau after roughly $\sim$1k SFT samples, and (ii) careful data selection or distribution-matched supervision can outperform much larger training pools (e.g., 5% subsets beating full-data baselines; 3×–4.5× data baselines being surpassed) [11, 12]. This motivates focusing on dataset choice and format alignment rather than maximizing dataset count, and it keeps ablations interpretable.

2.2.2 Experiment A: Sequential SFT

Setup. We start from the baseline SFT checkpoint and apply a *second* SFT stage in which the training mixture contains only Databricks Dolly 15k, `microsoft/orca-math-word-problems-200k`, and `AI-MO/NuminaMath-CoT`. The motivation was a curriculum-style schedule [9]: Dolly 15k provides broad instruction-following and conversational data, Orca-Math supplies grade-school word problems, and NuminaMath-CoT introduces harder high-school and olympiad-level problems, so that the model first consolidates basic math reasoning and is then exposed to more challenging exercises.

Results

Metric	Baseline SFT	Experiment A	Δ
ARC-Easy	0.5699	0.2517	-0.3182
ARC-Challenge	0.4121	0.2295	-0.1826
MMLU	0.3710	0.2380	-0.1330
GSM8K	0.0735	0.0000	-0.0735
HumanEval	0.0061	0.0000	-0.0061
SpellingBee	0.9961	0.0000	-0.9961
ChatCORE	0.3133	-0.0068	-0.3201

Table 3: Comparison of baseline SFT and Experiment A results. Δ indicates the change relative to the baseline.

Experiment A shows clear evidence of catastrophic forgetting [10]. After applying a second SFT stage using only Dolly 15k, Orca-Math, and NuminaMath-CoT, performance drops significantly across nearly all benchmarks. For example, ARC-Easy decreases from 0.5699 to 0.2517 and MMLU drops from 0.3710 to 0.2380. Several tasks collapse entirely, including GSM8K, HumanEval, and SpellingBee.

This behavior is consistent with prior continual-learning research showing that sequential training on disjoint datasets causes models to overwrite previously learned representations. Because the second SFT stage excludes the original training mixture (which contained datasets for reasoning, spelling, and conversational tasks), gradient updates during fine-tuning overwrite earlier capabilities. As a result, the model loses many previously learned skills despite being exposed to additional math datasets.

The complete collapse of GSM8K to 0.0000 can also be explained by format mismatch. GSM8K evaluation requires a specific output format, which the original SFT mixture explicitly trained. Orca-Math and NuminaMath-CoT use chain-of-thought with different answer delimiters, so after the second SFT stage the model's outputs no longer parse correctly, even if underlying numerical reasoning is partially intact.

2.2.3 Experiment B: Single mixed SFT

Setup. We start from the baseline pretrained checkpoint and run a single SFT stage using the original nanochat SFT mixture (SmolTalk, identity, spelling tasks, MMLU, ARC, GSM8K, SpellingBee) augmented with Databricks Dolly 15k and `whynlp/gsm8k-aug`. Based on the results of Experiment A and our literature review on catastrophic forgetting in sequential SFT, we deliberately use a *single* mixed SFT run rather than stacking a second stage, to avoid overwriting capabilities learned in the original SFT. We also remove NuminaMath-CoT and Orca-Math-200k from this experiment because their answer formats differ substantially from GSM8K, which previously caused the model to produce mismatched outputs and hurt GSM8K evaluation; in their place, we add `whynlp/gsm8k-aug`, an augmented GSM8K dataset that closely matches the target benchmark distribution and formatting.

Results.

Metric	Baseline SFT	Experiment B	Δ
ARC-Easy	0.5699	0.5690	-0.0009
ARC-Challenge	0.4121	0.4352	+0.0231
MMLU	0.3710	0.3702	-0.0008
GSM8K	0.0735	0.2449	+0.1714
HumanEval	0.0061	0.0427	+0.0366
SpellingBee	0.9961	0.9922	-0.0039
ChatCORE	0.3133	0.3520	+0.0387

Table 4: Comparison of baseline SFT and Experiment B results. Δ indicates the change relative to the baseline.

Experiment B substantially improves mathematical reasoning performance, with GSM8K increasing from 0.0735 to 0.2449 (+0.1714). HumanEval also improves from 0.0061 to 0.0427, suggesting some gains in code-related reasoning. ARC-Challenge shows a moderate improvement, while ARC-Easy and MMLU remain largely unchanged. The overall ChatCORE metric improves from 0.3133 to 0.3520, indicating better general chat performance. SpellingBee shows a small decrease, but remains near-perfect. Overall, the mixed SFT configuration improves reasoning benchmarks without significantly degrading general capabilities.

2.2.4 Conclusion

Overall, Experiment A shows severe catastrophic forgetting and large performance drops across benchmarks. Therefore, we proceed with Experiment B, which aims to incorporate additional datasets while preserving the original training mixture.

3 Part 3

3.1 Our Training Run vs. Original RL Run

The original run by Karpathy (Figure 1) was done on the original nanochat architecture (Karpathy, 2025) with a step size of roughly 450 steps. Our training run (Figure 2) was done on the nanochat baseline on the latest model architecture with roughly 950 steps. Karpathy's run was intentionally shorter as he noted that he did not spend much time on reinforcement learning as it had not been well fine-tuned and thus more a proof of concept rather than a full training run.

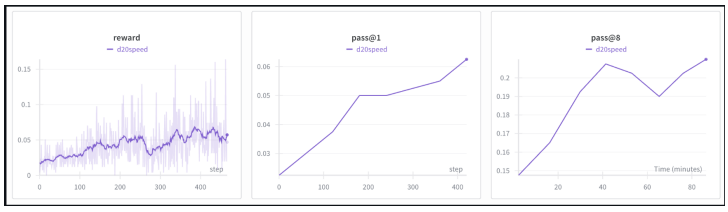

Figure 1: Karpathy's Original Run Graphs

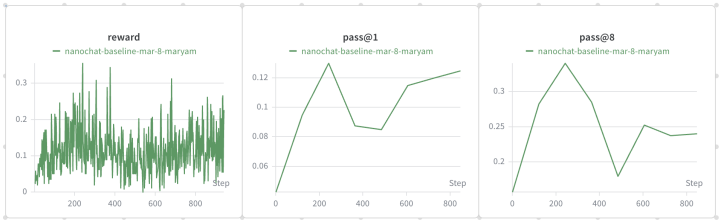

Figure 2: Our Training Run Graphs

We start by comparing the peak performance of the two models. Our nanochat model achieved roughly 2x the peak pass@1 (0.13 vs 0.065) and 55% higher peak pass@8 (0.34 vs 0.22). Our models better peak performance may have been a result of the architectural differences between the two models as our model, with the latest architecture, may have a greater reasoning capacity for the RL signal to build on. As it can be seen in the graphs, our peak performance occurs at around step 250, which is within the comparable window of Karpathy's run, and so is not a product of longer training.

Unlike the original run, our model experiences a sharp drop in pass@1 and pass@8 between steps 400 to 500. While this pattern is often associated with reward hacking, where the model has found a shortcut to score well on the reward function, it is difficult to say that this is the cause here as our reward function is a binary one, with 1.0 for correct answers and 0.0 for all false answers. This leaves little room for the model to game for partial credit. Thus, a more likely explanation is that the model underwent a period of policy instability, a known phenomenon in GRPO-style training where the model temporarily moves too far from its reference policy, or original model, causing a temporary degradation in output quality. Furthermore, our implementation removes KL regularization entirely, meaning there is no penalty pulling the model back toward its original behaviour. This makes the model more susceptible to large policy updates, as there is nothing explicitly constraining how far the weights can drift from the starting point. The partial recovery we observe after step 500 is therefore likely a product of the optimizer naturally finding its way back to a stable region rather than any explicit regularization mechanism. On the other hand, we do not see this occurring in Karpathy's original run. It is likely that his run simply ended before this instability had the chance to emerge, as Karpathy's training concluded at around step 450, which is around the onset of the region where our model begins to destabilize. Had his run continued, it is plausible that a similar period of policy instability would have occurred.

Despite the higher peak, our model's final pass@8 (0.24) converged close to the original's (0.22). This suggests the RL training on a general chat base reaches a similar steady-state to the original, even after more volatile training dynamics. The replication is therefore successful by final-step metrics, even if the path there was less stable and the training duration varied.

Moreover, in both runs, pass@8 far exceeds pass@1 throughout training. This gap indicates the model already "knows" the right answer in its distribution, as it can find it given enough samples, but struggles to reliably produce it on the first try. This is a strong signal that more RL epochs and more training steps would continue to pay off, by pushing pass@1 closer to the pass@8 ceiling.

Lastly, the nanochat reward curve is significantly noisier (raw spread ±0.18 vs ±0.10), reflecting that a general chat model has a wider output distribution. Small prompt variations produce bigger swings in answer quality early in training. This is expected behaviour, as a model trained broadly across many tasks will naturally exhibit higher variance when fine-tuned on a narrow domain, as the reward signal is pulling against a more diverse set of learned behaviours.

3.2 Cluster Analysis

Manual Analysis: Error Patterns by Calculation Step Count

To explore what types of problems the model struggled with, we categorised questions by the number of calculation steps required to reach the correct answer. Table 1 shows the total number of questions and incorrect answers for each step count.

Table 5: Questions and incorrect answers by number of calculation steps

Number of Calculation Steps	Total Questions	Total Answered Wrong
1	4	3
2	326	159
3	376	257
4	295	248
5	173	150
6	80	76
7	40	39
8	16	15
9	2	2
10	1	1
11	1	1

A clear trend emerges, error rate increases monotonically with the number of required calculation steps. Problems requiring only 2 steps had an error rate of 49%, while those requiring 6 or more steps were answered incorrectly nearly every time. This is consistent with the well-documented compounding error problem in language model reasoning, each intermediate step introduces a chance of error, and those errors propagate through subsequent steps, making multi-step problems disproportionately difficult.

Interestingly, 1-step problems showed a higher error rate (75%) than 2-step problems (49%), which warrants closer examination. Manual review of these 4 questions revealed a consistent pattern: in all cases, the model did not actually solve the problem in a single step as intended. In two of the four cases, the model began with the correct reasoning approach but made arithmetic errors in the final calculation. In the remaining cases, the model appeared to use the numbers from the problem statement without a coherent solution strategy, essentially combining figures arbitrarily rather than reasoning through the problem. None of the model's responses solved these problems in a single step, despite the problems being simple enough to require only one, the model consistently produced unnecessarily verbose or convoluted solutions, suggesting it has a tendency toward over-elaboration that can introduce errors even on straightforward problems.

Automated Cluster Analysis via LLM

To perform more detailed cluster analysis on the type of problems that were often being miscalculated, we fed our resulting json file into a Claude Sonnet 4.6 model and received the following results.

Table 6: Problem cluster analysis: error patterns by category

Problem Cluster	Count	Pass Rate	Primary Error Mode
Percentages & Fractions	361	23.3%	Misapplied fractions; dropped intermediate results
Time / Duration Arithmetic	200	19.0%	Treats clock times as plain integers; ignores AM/PM conversion
Multi-Stage / Cascading Logic	641	∼18%	Correct early steps, wrong operator or variable later in chain
Off-By-Small Arithmetic Errors	89	0%	Right approach, wrong arithmetic (e.g. 49−12=35 instead of 37)
Ratio / Proportion Problems	6	0%	Computes ratio denominator but skips applying correct fraction
Completely Wrong Approach	163	0%	Misreads problem setup; irrelevant arithmetic from the start

These results further tell us that the model's errors are not evenly distributed across problem types, but instead cluster around specific reasoning weaknesses. The largest category, Multi-Stage / Cascading Logic problems, aligns directly with the step-size analysis above, these are precisely the problems where compounding errors over multiple steps cause the model to fail. The model often executes the early steps correctly but applies the wrong operator or references the wrong intermediate variable later in the chain, suggesting it loses track of the problem state as reasoning depth increases.

Percentages and Fractions and Time/Duration Arithmetic represent the next most common failure modes. In both cases the errors are systematic rather than random, the model consistently misapplies fractional operations and treats clock times as plain integers, indicating these are learned bad habits rather than isolated mistakes. This is notable because it suggests targeted fine-tuning on these problem types could yield meaningful gains without requiring a full retraining run.

However, the most revealing clusters are the two categories with 0% pass rates: Off-By-Small Arithmetic Errors and Ratio/Proportion Problems. The former is particularly striking, the model demonstrates the correct approach and reasoning but fails at basic arithmetic, such as computing $49 - 12 = 35$ instead of 37. This class of error is unlikely to be resolved by more RL training alone, as the reward signal cannot easily distinguish a correct approach with a small arithmetic slip from a fundamentally wrong answer. The Completely Wrong Approach category (163 questions, 0% pass rate) is consistent with the 1-step anomaly identified earlier, where the model misreads the problem setup entirely and performs arithmetic on the numbers present without a coherent strategy.

Taken together, these two analyses show us that the model struggles most with problems that require sustained multi-step reasoning, and has specific systematic weaknesses around percentages, time arithmetic, and basic calculation accuracy.

4 Part 4

We deliberately chose simple reward functions over more complex alternatives. Our design decisions are supported by two key findings from recent literature.

Sahoo (2025) directly compared binary correctness rewards against multi-component continuous rewards for GRPO training on GSM8K. The binary reward achieved 40% accuracy and the continuous reward achieved 28%. The paper attributes this to the fact that additional reward components create proxy objectives that the model optimizes instead of actual correctness. They say "the continuous reward, despite richer information, may have provided conflicting objectives (e.g., encouraging fluency or verbosity) that diluted the focus on correctness" (Sahoo, 2025). Our two reward functions are intentionally lightweight (gives only 0.25, capped at 1.0) to avoid this failure mode while still providing mild structural guidance.

One natural alternative would be a distance-based reward (e.g., scaling reward by how numerically close the predicted answer is to the ground truth). We chose not to pursue this. Gao et al. (2024) demonstrate that richer, unbounded reward signals for math reasoning are prone to severe reward hacking. In their experiments, dense process rewards caused accuracy to collapse from 30.58% to 11.16%, far below even the pre-training baseline. A continuous partial-credit signal (rewarding proximity to the correct number) would similarly provide a dense, exploitable gradient that the model could hack.

4.1 Reward #1

Builds on the baseline by adding a format-shaping bonus of 0.25. A reward of 0.25 will be given if the model produces well-structured reasoning: the final line must begin with the #### answer marker, and the total response must contain between 2 and 8 newlines. The total is capped at 1.0.

The format bonus is intentionally small (0.25 vs 1.0 for correctness) since good formatting does not directly correlate with correctness. We did not want the model to be heavily rewarded for producing well-formatted but wrong answers. However, we hypothesize that encouraging structured output can indirectly promote more step-by-step reasoning. The newline range of 2–8 was chosen by examining the GSM8K dataset (OpenAI, 2021), where ground-truth solutions contain 2–8 reasoning steps, with each step generally separated by a newline. Encouraging a similar structure nudges the model toward multi-step reasoning rather than single-line answers. The #### marker requirement serves a practical purpose: since our correctness check relies on parsing the number after ####, rewarding its presence increases the chance that the model's answer can actually be evaluated. Without it, even a numerically correct response would receive zero correctness reward.

Pseudocode is below but can also be found in the gsm8k.py file, in our forked repository.

4.2 Reward #2

Builds on the baseline by adding a bonus of 0.25. All numbers are extracted from the user's question via regex. If at least 80% of those numbers appear somewhere in the model's response, the bonus is awarded. Capped at 1.0.

The reward is capped at 1.0 following the clipping principle from Gao et al. (2024), who show that unbounded reward accumulation leads to reward hacking. Without the cap, a model could inflate its score by simply listing many numbers. The 80% threshold and flat 0.25 bonus (rather than a continuous proportional score) further limit this: the model cannot incrementally farm reward by inserting more and more numbers.

Algorithm 1 Baseline + Format Reward

function REWARD(conversation, response)
    $r \leftarrow 0.0$
    $\hat{y} \leftarrow$ EXTRACTANSWER(response)
    $y \leftarrow$ EXTRACTANSWER(conversation)
    if $\hat{y} = y$ **then**
        $r \leftarrow r + 1.0$ ▷ Correctness reward
    end if
    $\ell \leftarrow$ last line of response (stripped)
    $n \leftarrow$ number of newlines in response
    hasMarker $\leftarrow \ell$ starts with `"####"`
    goodLength $\leftarrow 2 \leq n \leq 8$
    if *hasMarker* $\wedge$ *goodLength* **then**
        $r \leftarrow r + 0.25$ ▷ Format bonus
    end if
    return $\min(r,\ 1.0)$
end function

This reward could still be hacked. A model could learn to dump a list of numbers from the prompt into its response without performing genuine reasoning, satisfying the 80% threshold cheaply. This is a limitation that might reduce the effectiveness of this bonus in practice.

We considered an alternative design that checks whether numbers from the ground-truth solution appear in the response. However, we realized if the model reproduces all numbers from the ground truth, it almost certainly has the final answer correct and would already receive 1.0 from the correctness reward. The bonus would therefore never provide a useful additional signal for partially-correct responses. By checking against the prompt numbers instead, we reward the model for explicitly referencing the given quantities in its reasoning, which can benefit responses that set up the problem correctly but arrive at the wrong final answer.

Pseudocode is below but can also be found in the gsm8k.py file, in our forked repository.

Algorithm 2 Baseline + Reference Numbers Reward

function REWARD(conversation, response)
    $r \leftarrow 0.0$
    $\hat{y} \leftarrow$ EXTRACTANSWER(response)
    $y \leftarrow$ EXTRACTANSWER(conversation)
    if $\hat{y} = y$ **then**
        $r \leftarrow r + 1.0$ ▷ Correctness reward
    end if
    $Q \leftarrow$ user question from conversation
    $\mathcal{N} \leftarrow$ EXTRACTALLNUMBERS(Q) ▷ via regex `\d+\.?\d*`
    if $j\mathcal{N}j$>0 **then**
        $m \leftarrow j\,\{n \in \mathcal{N} \mid n \in \text{response}\}\,j$ ▷ Count matched numbers
        if m / $j\mathcal{N}j \geq 0.8$ **then**
            $r \leftarrow r + 0.25$ ▷ Reference numbers bonus
        end if
    end if
    return $\min(r,\ 1.0)$
end function

4.3 Table of Results

NOTE: Please refer to the Appendix for the graphs from WandB.

Table 7: Final Evaluation Scores (Post-RL)

Benchmark	Baseline	Reward 1 (Format)	Reward 2 (Ref Numbers)	Both (R1+R2)
GSM8K	0.1501	**0.3124**	0.2631	0.2980
ARC-Easy	**0.5673**	0.5417	0.5665	0.5425
ARC-Challenge	0.4258	**0.4275**	0.4181	0.4266
MMLU	0.3675	**0.3695**	0.3668	0.3638
HumanEval	0.0183	0.0183	0.0244	**0.0549**
SpellingBee	0.0000	0.0000	0.0000	0.0000
ChatCORE	0.1638	0.1859	0.1816	**0.1883**

Table 8: Training Dynamics (from W&B Logs)

Metric	Baseline	Reward 1 (Format)	Reward 2 (Ref Numbers)	Both (R1+R2)
Starting pass@1 (step 0)	0.043	0.165	0.165	0.165
pass@1 at step 120	0.130	0.228	0.233	**0.245**
pass@1 at step 240	0.085	**0.290**	0.208	0.263
pass@1 at step 300	0.115	**0.308**	0.222	0.278
Final pass@1 (step 420)	0.125	**0.302**	0.238	0.290
Final pass@8	0.180	**0.433**	0.350	0.430
Avg. sequence length trend	$238 \rightarrow 130$	$174 \rightarrow 100$	$174 \rightarrow 90$	$174 \rightarrow 130$

Table 9: pass@1 to pass@8 Gap Over Training

Configuration	Gap at Step 0	Gap at Step 420
Baseline	0.114	0.055
Reward 1 (Format)	0.268	0.143
Reward 2 (Ref Numbers)	0.268	0.112
Both (R1+R2)	0.268	0.140

4.4 Analysis of Results

Reward 1 (Baseline + Format) performed the best overall on mathematical reasoning. It more than doubled GSM8K accuracy over the baseline (0.3124 vs 0.1501), with consistent improvement throughout training. The format bonus appears to have had two beneficial effects. First, encouraging the #### marker ensured the model's answers were actually parseable by the correctness checker, meaning correct reasoning was no longer wasted on unparseable outputs. Second, the 2–8 newline constraint nudged the model toward multi-step chain-of-thought reasoning rather than single-line guesses. Importantly, non-GSM8K benchmarks (ARC, MMLU) were unchanged, suggesting the format reward did not cause deterioration or forgetting. The combined reward (Both) started even stronger than Reward 1 at step 120 (0.245 vs 0.228), but Reward 1 overtook it by step 240 (0.290 vs 0.263) and maintained the lead through step 420 (0.302 vs 0.290). This crossover suggests the format reward is the more effective driver of late-stage mathematical improvement, while the combined reward's early speed advantage comes from its denser gradient signal.

Pass@1 is the accuracy when the model gets one attempt. Pass@8 is the accuracy when it gets 8 attempts and any one being correct counts. A larger pass@8 means the model can solve more problems in principle, but doesn't reliably pick the right approach on the first try. So outputs are diverse, and some of that diversity is correct. At step 420, Reward 1 has pass@1 = 0.302 but pass@8 = 0.433, a gap of 0.143. This means there are ∼13% more problems the model can solve but doesn't consistently nail on the first attempt. It learned multiple valid reasoning strategies rather than collapsing to one rigid pattern. Both (R1+R2) has a gap of 0.140, nearly identical to Reward 1, suggesting that adding the reference number reward does not meaningfully reduce the model's behavioral diversity. Reward 2's gap is smaller (0.112), meaning it converged more aggressively to a narrower set of behaviors. Reward 1 made the model both better and more flexible.

Reward 2 (Baseline + Reference Numbers) improved over baseline but underperformed Reward 1. GSM8K rose to 0.2631, which is a 75% improvement over baseline, but still 16% below Reward 1. A likely explanation is the reward hacking vulnerability we identified. The model can satisfy the 80% threshold by echoing prompt numbers without performing genuine multi-step reasoning. All three trained models produce shorter outputs over time, but Reward 2 drops to ∼90 tokens the fastest, compared to Reward 1 (∼100 tokens) and Both (∼130 tokens), suggesting the model might have learned to produce brief responses that mention prompt numbers without extended reasoning. Notably, the pass@1 curve for Reward 2 plateaued around step 120 (∼0.233) and showed little further improvement, whereas the model with both rewards improved past step 120 and reached 0.245 at that same checkpoint. This suggests the format reward component in the combined model acts as a corrective signal that prevents the early stagnation caused by Reward 2 hacking alone.

Reward 2 is also the only run where pass@8 actually declines over training from 0.4325 at step 0 to 0.3700 at step 420 (net −0.063). The model becomes less capable with unlimited attempts, not just less consistent. Model with both rewards nearly preserves pass@8 (0.4325 → 0.4300, net −0.003), essentially flat throughout. Reward 1 modestly improves it (+0.013). This means the format reward component in the combined reward model is doing real work as a stabilizing force—without it, the reference number reward hacking actually erodes the model's underlying problem-solving repertoire.

The baseline reward alone was insufficient. Despite training for the same number of steps, the baseline struggled to consistently improve, with pass@1 fluctuating between 0.085–0.130 after step 120. With the baseline, all wrong answers look identical (reward = 0.0), so the model gets no signal about how to improve when it fails. This is especially important early in training when most answers are wrong.

The combined reward run raises an important point about reward signal density versus reward-task alignment. Per-batch rewards for Both averaged 0.53–0.62 across training (substantially higher than Reward 1's ∼0.23–0.36, since both reward components can be earned independently on every answer), providing extremely dense gradient signal. Yet this did not translate to better final GSM8K performance than Reward 1 alone. So what matters is whether the reward directly incentivizes the target capability. Both rewards do reward correct answers, but the reference number component adds a second optimization target that partially dilutes the model's focus on pure reasoning quality. The result is a run that learns quickly early (denser signal, faster initial gradient updates) but plateaus below Reward 1's ceiling.

No reward configuration harmed the non-math benchmarks, with one notable exception in the positive direction. All configurations maintained similar ARC-Easy (∼0.54–0.57), ARC-Challenge (∼0.42–0.43), and MMLU (∼0.37) scores. However, Both (R1+R2) achieved the best HumanEval score by a substantial margin (0.0549 vs 0.0244 for Reward 2 and 0.0183 for the others), and the best ChatCORE score (0.1883 vs 0.1859 for Reward 1). The HumanEval improvement is particularly striking—more than 2× better than any single reward. A plausible explanation is that the combination of structured output formatting (R1) and explicit reference to numeric values in context (R2) generalizes well to code generation, where both properties are directly useful: code benefits from structured layout, and algorithmic problems require correctly using input values. This suggests that while R1+R2 is not the best combination for pure math reasoning, the two rewards may be complementary for tasks requiring structured, value-aware outputs.

Additionally, the baseline's per-batch rewards are extremely low throughout training (mostly 0.02–0.17 in early steps), meaning most batches carry near-zero signal for GRPO to learn from. Rewards 1 and 2 start at ∼0.23–0.36 per batch because even wrong answers can earn the 0.25 format or reference bonus. Both starts even higher (∼0.53) for the same reason. This denser signal means more informative gradient updates per step. The baseline's pass@1 drops from 0.130 (step 120) to 0.085 (step 240) before recovering to 0.115 (step 300). This oscillation is consistent with a sparse binary reward where too few positive signals per batch leads to noisy, unreliable gradient updates. All three non-baseline runs avoid this regression. All

configurations generally plateau in the final ∼120 steps as the learning rate decays linearly to near-zero (∼0.002 by step 466), limiting further updates regardless of reward quality.

Note: The baseline started from model step 500, while Rewards 1, 2, and combined reward models started from step 534 (more SFT data). This gives the latter three a head start (pass@1 = 0.165 vs 0.043 at step 0). The GSM8K improvements are still meaningful since all four configurations trained for the same number of RL steps.

4.5 Visualizations

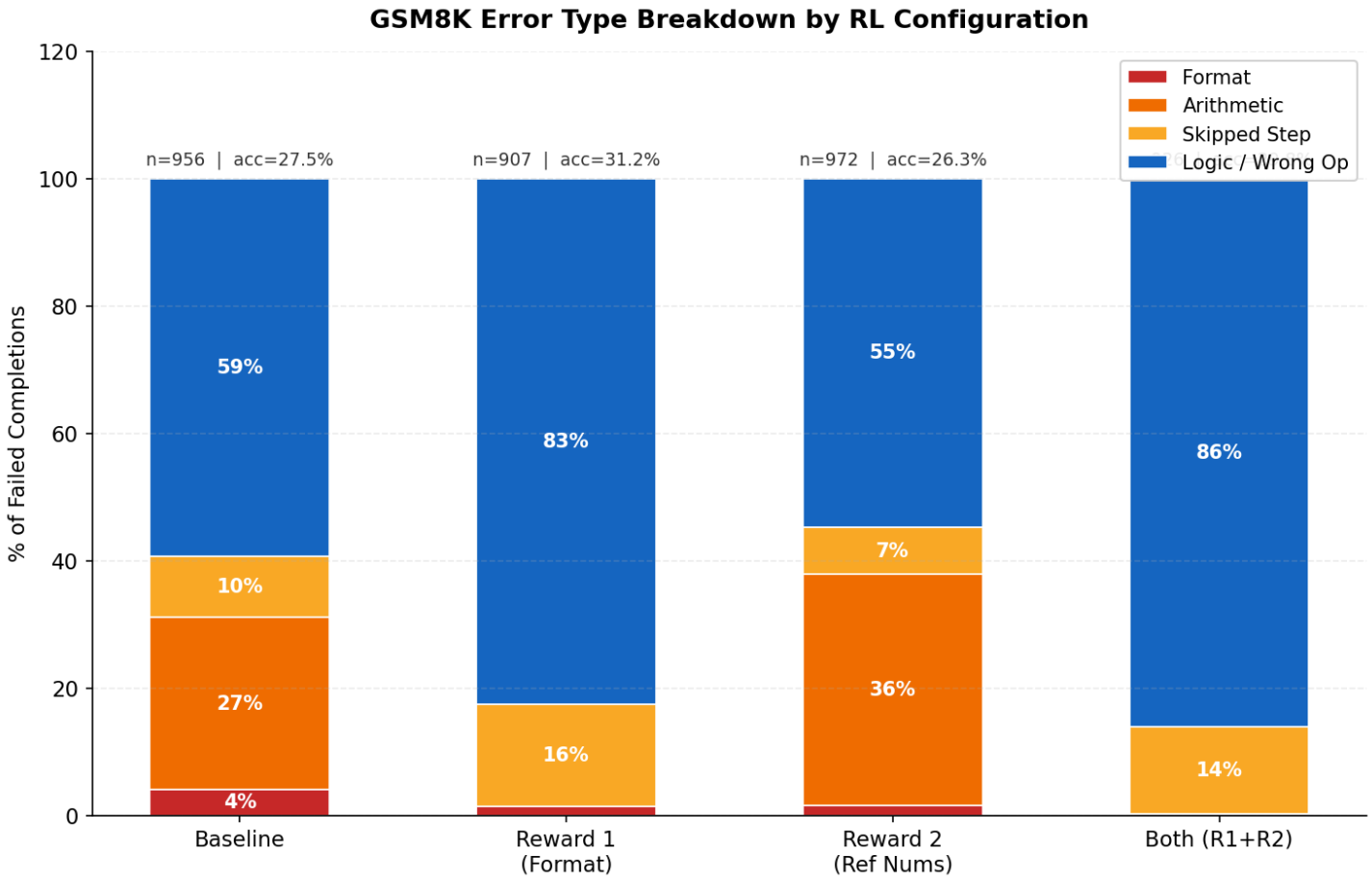

Figure 3: Error type breakdown for each RL configuration across all 1,319 GSM8K failures.

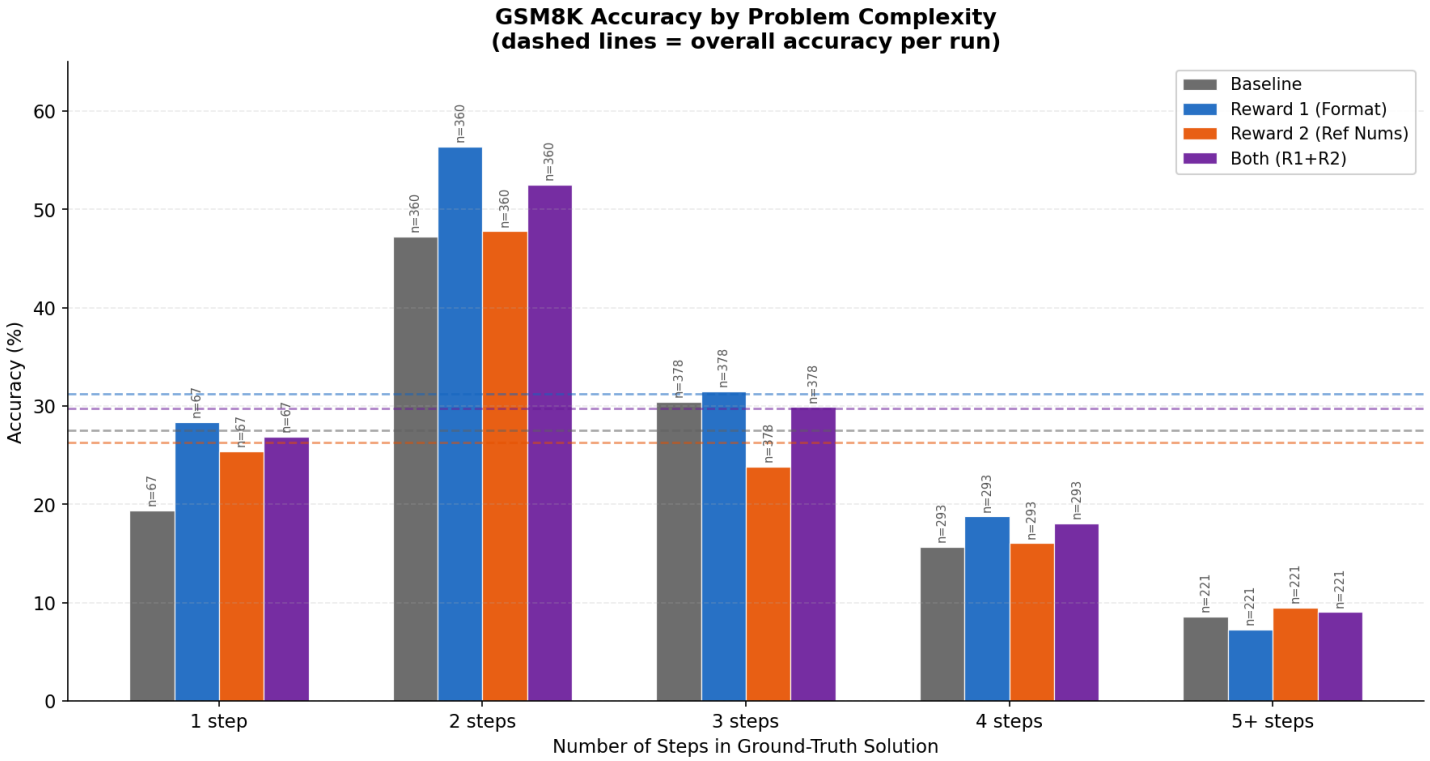

Figure 4: GSM8K accuracy binned by the number of arithmetic operations in the ground-truth solution. Dashed horizontal lines mark each run's overall accuracy as a reference.

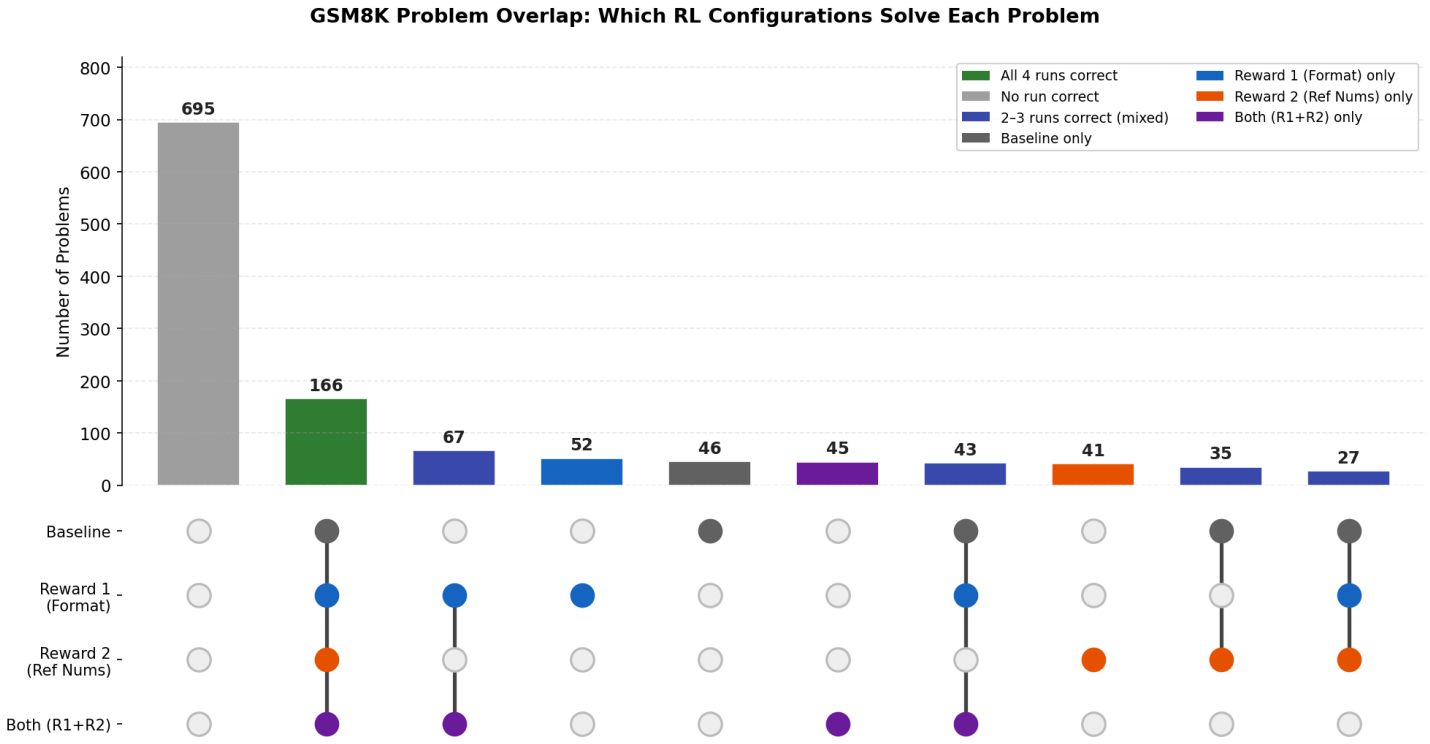

Figure 5: UpSet-style overlap of correctly solved GSM8K problems across all four RL configurations.

Figure 1 decomposes every failed GSM8K completion into one of four error types: Format (missing #### marker), Arithmetic (at least one wrong result inside a <<>> tag), Skipped Step (significantly fewer operations than the ground-truth solution), and Logic / Wrong Operation (correct structure and arithmetic, but wrong reasoning). It plots their relative proportions as a stacked bar for each configuration. The most important takeaway is the complete elimination of arithmetic errors in Reward 1 and Both, because both runs use python tool-call tokens exclusively, the runtime always computes the correct result, reducing their arithmetic error share to zero. In contrast, 27% of Baseline failures and 36% of Reward 2 failures involve at least one wrong <<>> calculation—errors that cascade through subsequent steps and compound into wrong final answers. Reward 2 is actually worse than the Baseline on arithmetic errors despite having an explicit reward signal, without pressure to maintain the multi-line tool-call style, R2 collapsed to plain-text calculator notation that forces the model to predict numerical tokens by autocomplete. The figure also reveals that Reward 1 and Both have nearly identical error distributions (83% vs. 86% pure logic). The practical implication is that roughly 37–43% of Baseline and R2 failures are structurally avoidable: those problems are within the model's reasoning ability, but arithmetic slippage in intermediate steps prevents correct final answers.

Figure 2 plots GSM8K accuracy for each configuration against the number of arithmetic operations in the ground-truth solution (binned as 1, 2, 3, 4, and 5+ steps), with dashed horizontal lines marking each run's overall accuracy as a reference. The figure exposes two distinct patterns. First, all four configurations peak sharply at 2-step problems (47–57% accuracy) and then decline monotonically—confirming that problem complexity is a hard universal ceiling for this model size, independent of reward design. Second, the configurations diverge at exactly the point where multi-step reasoning begins to matter. Reward 2's accuracy at 3-step problems (24%) falls below the Baseline (30%), meaning the R2 actively degrades performance on moderately complex problems. Reward 1 and Both remain above their own dashed overall-accuracy lines at 2 and 3 steps, meaning they over-perform their averages on medium-complexity problems since the structured, multi-line format they use provides enough scaffolding to handle these cases better than their global numbers would suggest. By 4 steps, all shaped-reward runs converge near the Baseline, and at 5+ steps all four runs bottom out at 8–10%, revealing a model-level reasoning ceiling that no reward design at this training scale can overcome.

Figure 3 uses an UpSet-style plot to show how the 1,319 GSM8K problems distribute across the 10 most common combinations of which runs get each problem correct. The bars are sorted by frequency and the dot matrix below identifies which configurations are "in" each group. The most important number is the leftmost bar: 695 problems (53%) are solved by no configuration, a hard lower bound on the fraction of problems that exceed this model's reasoning capacity regardless of reward design, and a reminder that all four runs are working within the same fundamental model limitations. The second bar—166 problems solved by all four—represents the easy floor where reward design is irrelevant. Between these extremes, the figure reveals the genuine differentiation: 67 problems are solved by Reward 1 and Both but not by Reward 2 or Baseline, the clearest signature of the format reward's unique contribution; these are problems where having access to the python calculator tool was necessary and neither <<>> arithmetic nor the base model was sufficient. Reward 1 uniquely solves 52 problems no other run gets, and Both uniquely solves 45—showing that the two runs, while correlated, are not identical and the combined reward does produce some problems that R1 alone misses, consistent with the HumanEval and ChatCORE advantage discussed later. Notably, Reward 2 uniquely solves 41 problems, confirming that despite its weaker overall accuracy, the reference-number reward does unlock a distinct subset of problems—likely those with straightforward sequential computation where echoing prompt numbers is sufficient. The overlap plot thus answers the question of whether the rewards produce genuinely different capabilities or whether one strictly dominates: R1 is the strongest single reward, but neither R2 nor Both is strictly dominated, and the combined run's unique-solve set (45 problems) points to real complementarity on a small but non-negligible fraction of the evaluation set.

4.6 Mistakes

NOTE: You can find the json files of all the questions and responses in the RL evaluation in our repository under the /eval_logs directory.

The most significant difference between Reward 1 and Reward 2 is the output format each model learned to produce, which has a direct and substantial impact on arithmetic accuracy. During supervised fine-tuning (SFT), the GSM8K training data was preprocessed to convert the dataset's original <<>> calculator annotations into the model's built-in tool-call token format:

<|python_start|>expression<|python_end|><|output_start|>result<|output_end|>.

When the model emits a <|python_start|> token during inference, the `nanochat` engine intercepts the generation, accumulates the expression tokens until <|python_end|>, executes the expression through a sandboxed Python `eval()`, and force-injects the correct result between <|output_start|> and <|output_end|> tokens so the model never predicts the arithmetic result, the runtime computes it.

Reward 1's format bonus encouraged the model to maintain the multi-line, natural-language-with-tool-calls style it learned during SFT, preserving its access to this calculator tool. Reward 1's completions consistently use this format (1,319 tool-call invocations across the evaluation set), meaning its arithmetic is always correct. Its errors are exclusively logical (choosing the wrong operation or misunderstanding the problem). For instance, on Problem 29 (board cutting), Reward 1 incorrectly divides by 3 instead of 5 ($40/3 = 13.33$), and on Problem 31 (seasonal elves), it subtracts from 20 instead of 60 ($20 - 10 = 10$

rather than $60 - 20 = 40$, $40 - 10 = 30$). Both are reasoning errors, not arithmetic errors, as the tool computed each expression correctly.

Reward 2, which had no format constraint, the model used the shorter <<>> format because it requires fewer tokens per response, aligning with the sequence length compression we observed in training (Reward 2's responses shrank to ~ 90 tokens vs Reward 1's ~ 100). However, these <<>> tags are plain text with no special handling in the `nanochat` engine so he model must predict both the expression and its numerical result as tokens, performing arithmetic by autocomplete.

This leads to frequent hallucinated arithmetic errors that Reward 1 never makes: on Problem 11 (fruit counting), Reward 2 writes <<3+5+6=15>> when $3+5+6 = 14$; on Problem 25 (aquarium shopping), it writes <<2.50*2.50=6.00>>, squaring 2.50 instead of computing $2 \times 2.50 = 5.00$; on Problem 32 (candy change), it writes <<3.5+7.5=12>> when $3.5 + 7.5 = 11$; and on Problem 36 (house values), it writes <<76000*.30=22400>> when $76,000 \times 0.30 = 22,800$. In the entire evaluation set, Reward 2 only emitted the <|python_start|> token twice, and both instances were garbled fragments where the token appeared mid-expression inside a <<>> tag (e.g., <<4500/ kW=<|python_start|>4500/ kW<|python_end|>) rather than as an intentional tool invocation.

Reward 2 frequently collapses multi-step problems into too few reasoning steps, dropping intermediate operations that are critical to arriving at the correct answer. On Problem 38 (fence perimeter), Reward 2 produces a single calculation <<20*15=300>>— computing the rectangle's area rather than its perimeter — and outputs #### 300, while Reward 1 correctly lays out three separate steps: 20*2=40, 15*2=30, 40+30=70. On Problem 40 (seashells), Reward 2 computes <<10*3/5=6>><<6*2=12>>, arriving at 12 by forgetting to multiply by the 9 people in each group — a step that is simply absent from the response. On Problem 13 (weekend movies), Reward 2 writes <<4/2=2>><<4+2=6>><<6+4=10>>, adding 4 instead of multiplying by 4 weeks, likely because the compressed format provides no natural-language context (such as "over 4 weeks") to distinguish between the number 4 appearing as a quantity to add versus a multiplier. This pattern suggests that Reward 2's terse, calculator-only format discourages the model from explicitly reasoning about what each number represents before computing with it — without natural language scaffolding between steps, the model is more likely to apply the wrong operation or skip steps entirely, because there is no intermediate text forcing it to articulate the relationship between quantities.

Reward 1 generates natural-language reasoning interleaved with tool-call tokens. For example, on Problem 6 (gorilla bananas), it outputs: She had 48/2=<|python_start|>48/2<|python_end|><|output_start|>24.0<|output_end|>24 bananas", followed by subsequent lines like "He had 24+25=...49 bananas" and "He had 49-12=...37 bananas." Each step is a complete English sentence that names the quantity being computed, followed by a tool-invoked calculation. Reward 1's natural-language scaffolding serves as a form of implicit chain-of-thought, forcing the model to articulate what each number represents before computing with it.

Despite its tool-call advantage, Reward 1 occasionally ignores its own tool output and substitutes an incorrect value. On Problem 39 (car fleet), the tool correctly computes 240000*.1=24000.0, but the model writes "2400" in the subsequent step, overriding the injected result. This likely occurs because the model generates the text *after* the <|output_end|>token autoregressively — while the tool forces the correct result into the token stream, the model must still attend to that result when constructing the next line, and sometimes fails to do so, possibly defaulting to a round-number approximation from its own internal representation rather than faithfully copying the tool output.

In summary, the two reward functions produce qualitatively different failure modes. When Reward 1 is wrong, it tends to have the right *structure* but applies the wrong operation or misinterprets the problem. On Problem 31 (seasonal elves), Reward 1 sets up two steps as expected but subtracts from 20 instead of 60; on Problem 29 (board cutting), it divides by 3 instead of 5. The reasoning chain *looks* correct at a glance and the error is semantic, not structural. Reward 2's failures are more varied and more fundamental: it may hallucinate arithmetic within <<>>tags, or apply the wrong operation.

Reward 2 does outperform Reward 1 on a subset of problems that map cleanly to a short, sequential chain of operations with no ambiguity about what to compute at each step. On Problem 29 (board cutting), Reward 2 concisely and correctly produces <<40/5=8>><<8*4=32>>, while Reward 1 divides by 3 instead of 5 despite having access to the calculator tool. On Problem 31 (seasonal elves), Reward 2 correctly chains three operations — <<60/3=20>><<60-20=40>><<40-10=30>>— while Reward 1 skips the subtraction from 60 entirely, going straight from 60/3=20 to 20-10=10. In these cases, the problem structure is straightforward enough that natural-language scaffolding provides no additional benefit: the numbers in the problem statement directly correspond to operands, and the operations follow in the same order as the narrative. Reward 2's terse format may even be advantageous here, as the model commits to a clean computation chain without the overhead of generating filler text that could introduce confusion. However, these problems represent the minority. When the mapping between the problem narrative and the required operations is less direct, requiring the model to reason about *which* operation to apply or to track multiple interacting quantities, Reward 2's lack of natural-language context becomes a liability rather than an efficiency.

Despite their differences, both reward functions fail on the same categories of conceptually difficult problems, suggesting these failures stem from the underlying model's reasoning limitations rather than from the reward design. The first category is directional or relative problems, where the model must recognize that two quantities work in opposition. On Problem 33, "Elvis starts driving from his house and travels west for 5 hours. Then he turns around and travels east for 8 hours. If he was driving at an average speed of 18mph for both parts of the journey, how far is he from his house now?" — both models add the total hours ($5 + 8 = 13$) and multiply by speed ($13 \times 18 = 234$), failing to recognize that traveling east partially

cancels the westward distance and that the correct approach is to compute the difference ($8 \times 18 - 5 \times 18 = 54$). The second category is complex word problems involving many entities, unit conversions, or conditional logic. On Problem 1, "Lorraine and Colleen are trading stickers for buttons. Each large sticker is worth a large button or three small buttons. A small sticker is worth one small button. A large button is worth three small stickers..." — both models fail to correctly track the exchange rates across multiple trading steps. The third category is percentage problems requiring multi-step reasoning. On Problem 27, "Jerry is rolling a six-sided die. How much more likely is it (expressed as a percentage) that he rolls a number greater than 3 than that he rolls two even numbers in a row?" — both models fail to correctly compute and then compare two separate probabilities. These shared failure modes indicate that neither reward function can compensate for the model's fundamental difficulty with problems requiring abstract relational reasoning, multi-entity tracking, or compositional understanding of percentage relationships.

The combined reward run (Both) exhibits a mistake profile that closely mirrors Reward 1 rather than Reward 2, with one important qualification. Despite Reward 2's influence during training, Both uses the python tool-call format in 100% of its completions. This alone accounts for a substantial portion of Both's accuracy advantage over the baseline.

The baseline's mistake profile is structurally identical to Reward 2's: all completions use the <<`expression=result`>> format, meaning the model must predict both the expression and its result as plain tokens. Arithmetic errors are pervasive and cumulative. On Problem 46 (Nani's ages), the baseline makes three sequential arithmetic errors: treating 25% as the literal decimal 0.25 (computing $8 - 0.25 = 7$ instead of $8 - 2 = 6$), then predicting $8 + 16 + 7 = 37$ (actual: 31), then predicting $8 + 37 = 48$ (actual: 45). Combined reward model, by contrast, makes a single logical error on the same problem—applying the 25% to the brother's age (16) instead of Nani's age (8)—and its arithmetic is correct throughout.

Missing #### markers are not a meaningful failure mode for the baseline in this evaluation. The marker appears in nearly all of the first 50 failures; the rare missing-marker cases across the full dataset (3% of entries) are attributable to mid-generation truncation artifacts, not cases of correct reasoning without an answer. The baseline's failures are genuinely wrong answers, not correct reasoning that went undetected.

The combined reward model's failure mode is almost exclusively logical. On Problem 21 (Tracy's wire), Both attempts to convert feet to inches but divides instead of multiplies ($4 \div 12 = 0.333$), then the model predicts the display token after the runtime output as `6`—a hallucinated value with no relationship to the computed result. When the Python runtime returns a non-integer float (e.g., `0.3333...`), the model must still predict a display token after <|`output_end`|>, and it occasionally predicts a plausible-looking but incorrect integer. Across the full Both evaluation, 268 of the tool call outputs involve a mismatch between the runtime result and the model's subsequent display token, of which 189 (70.5%) are float-to-integer truncations (benign) and 79 (29.5%) are larger discrepancies including digit drops and sign flips (harmful). This suggests an imperfect coupling between the tool output and the model's subsequent token prediction—the model sometimes fails to faithfully attend to the injected result before continuing generation.

Of the 30 problems where both the baseline and combined reward model fail, only two produce the same wrong answer via the same reasoning path. On Problem 33 (Elvis driving), both models add the total travel time ($5 + 8 = 13$ hours) and multiply by speed ($18 \times 13 = 234$), failing to recognize that eastward travel partially cancels the westward distance. On Problem 23 (Ellen's carrot cost), both correctly compute the cost of two carrots but stop there, returning 2 instead of dividing by 2 to get the cost per carrot. For the remaining 28 co-failures, the two runs make different errors entirely, the baseline reaches wrong answers through incorrect arithmetic on correct reasoning chains, while Both reaches different wrong answers through incorrect reasoning on correct arithmetic chains—producing divergent final values from the same starting problem.

The combined reward run thus inherits the best structural property of Reward 1 (tool-call format preservation, zero arithmetic errors) while introducing one new failure mode unique to that format (hallucinated display tokens for non-integer outputs) and retaining the same logical error profile as Reward 1. The baseline and Reward 2, sharing the <<>> format, are both exposed to arithmetic errors on top of whatever logical errors they make. The key implication is that the format reward is doing something more fundamental than enforcing output style: it is determining whether the model has access to exact arithmetic, which propagates into every subsequent reasoning step.

References

[1] Sahoo, S. (2025). The Good, The Bad, and The Hybrid: A Reward Structure Showdown in Reasoning Models Training. *ARLET Workshop, NeurIPS 2025*. https://arxiv.org/abs/2511.13016

[2] Gao, J., Xu, S., Ye, W., Liu, W., He, C., Fu, W., Mei, Z., Wang, G., & Wu, Y. (2024). On Designing Effective RL Reward at Training Time for LLM Reasoning. *arXiv preprint arXiv:2410.15115*. https://arxiv.org/abs/2410.15115

[3] OpenAI. (2021). GSM8K: Grade School Math 8K. *Hugging Face Datasets*. https://huggingface.co/datasets/openai/gsm8k

[4] Karpathy, A. (2025). Introducing nanochat: The best ChatGPT that $100 can buy. *GitHub*. https://github.com/karpathy/nanochat/discussions/1

[5] GSM8K-AUG dataset. https://huggingface.co/datasets/whynlp/gsm8k-aug

[6] Databricks Dolly 15K dataset. https://huggingface.co/datasets/databricks/databricks-dolly-15k

[7] Orca Math Word Problems 200K dataset. https://huggingface.co/datasets/microsoft/orca-math-word-problems-200k

[8] NuminaMath-CoT dataset. https://huggingface.co/datasets/AI-MO/NuminaMath-CoT

[9] Soviany, P., Ionescu, R. T., Rota, P., & Sebe, N. (2021). Curriculum Learning: A Survey. *arXiv preprint arXiv:2101.10382*. https://arxiv.org/abs/2101.10382

[10] van de Ven, G. M., Soures, N., & Kudithipudi, D. (2024). Continual Learning and Catastrophic Forgetting. *arXiv preprint arXiv:2403.05175*. https://arxiv.org/abs/2403.05175

[11] Dong, G., Yuan, H., Lu, K., Li, C., Xue, M., Liu, D., Wang, W., Yuan, Z., Zhou, C., & Zhou, J. (2023). How Abilities in Large Language Models are Affected by Supervised Fine-tuning Data Composition. *arXiv preprint arXiv:2310.05492*. https://arxiv.org/abs/2310.05492

[12] Xia, M., Malladi, S., Gururangan, S., Arora, S., & Chen, D. (2024). LESS: Selecting Influential Data for Targeted Instruction Tuning. *arXiv preprint arXiv:2402.04333*. https://arxiv.org/abs/2402.04333

5 Appendix

1 Basline SFT run

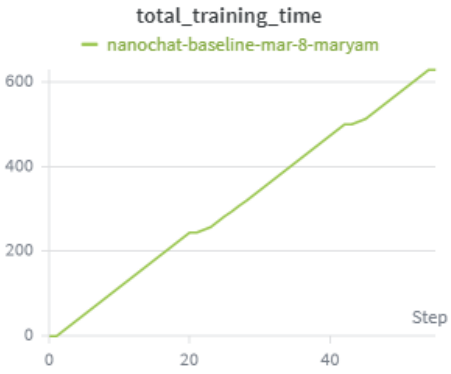

Figure 1:

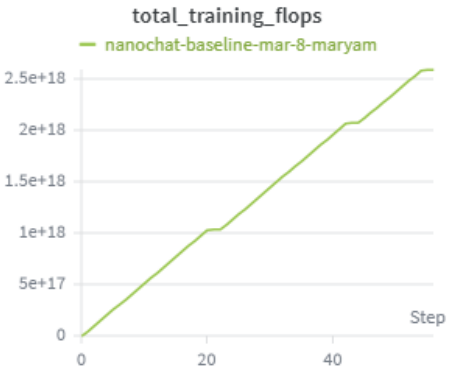

Figure 2:

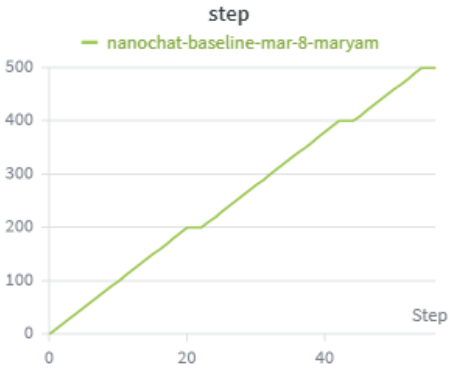

Figure 3:

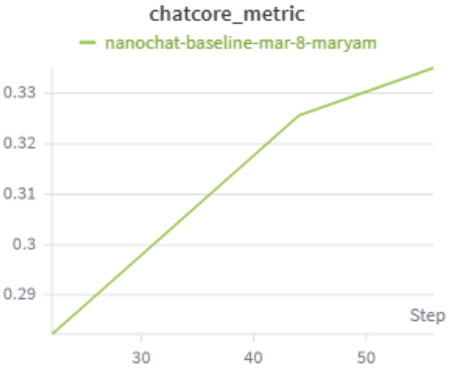

Figure 4:

References

[1] Weights and biases.

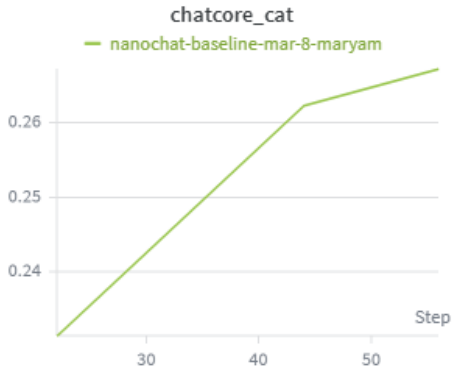

Figure 5:

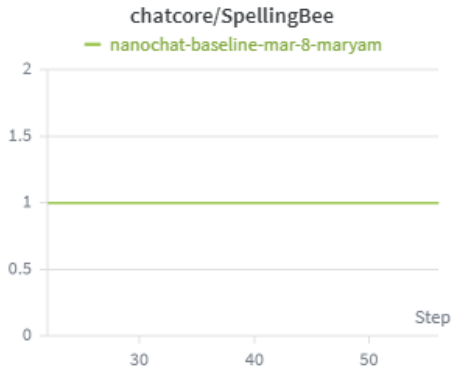

Figure 6:

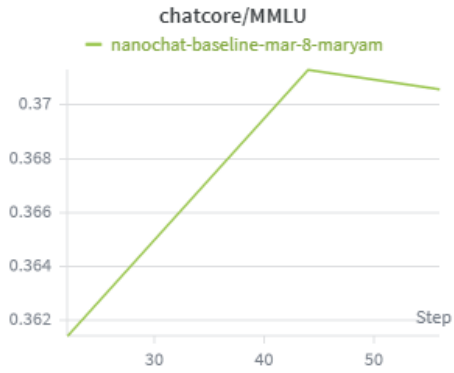

Figure 7:

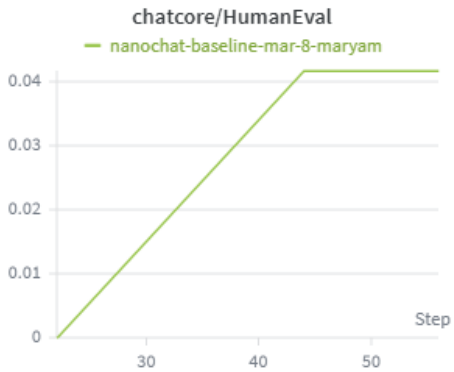

Figure 8:

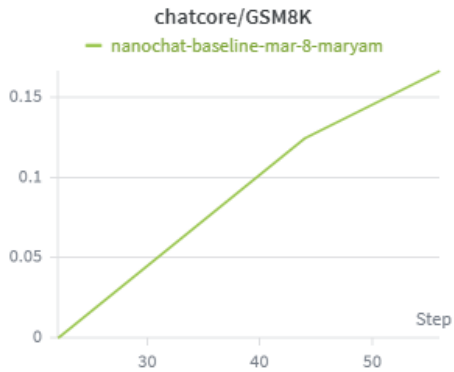

Figure 9:

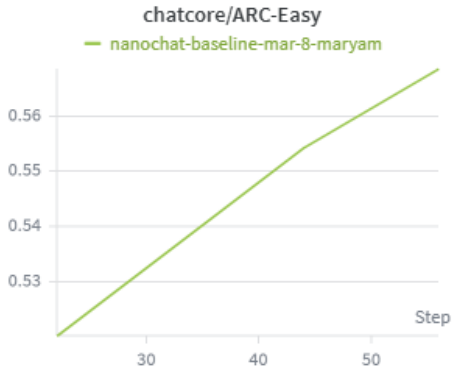

Figure 10:

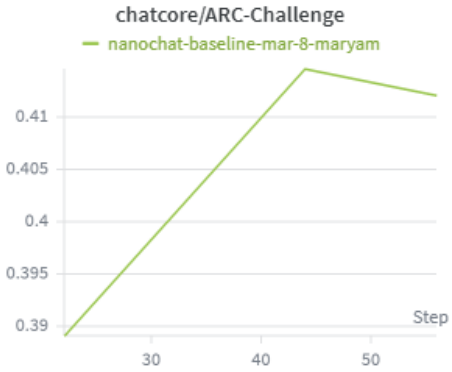

Figure 11:

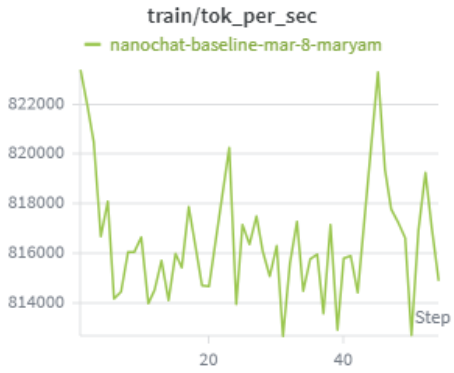

Figure 12:

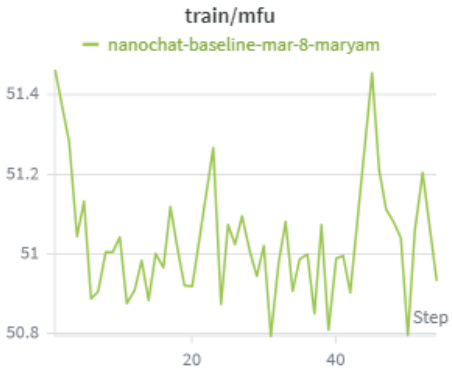

Figure 13:

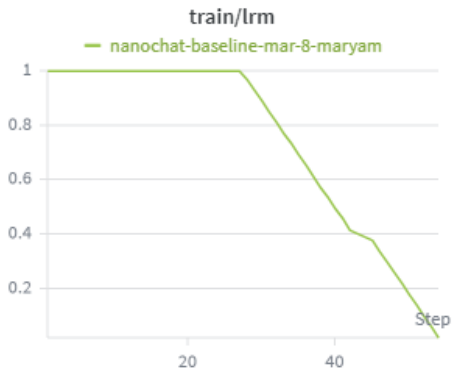

Figure 14:

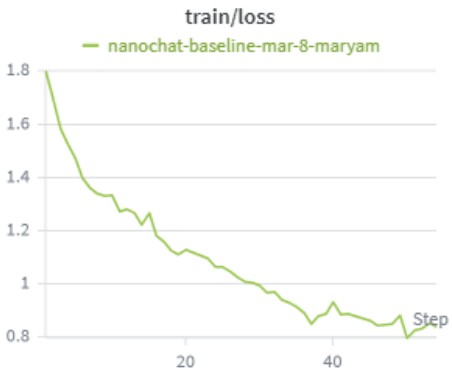

Figure 15:

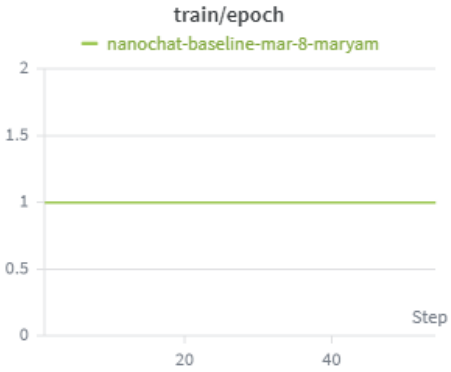

Figure 16:

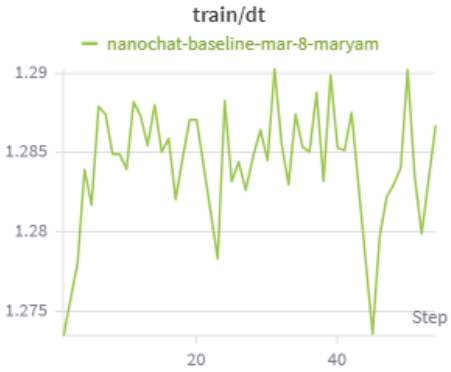

Figure 17:

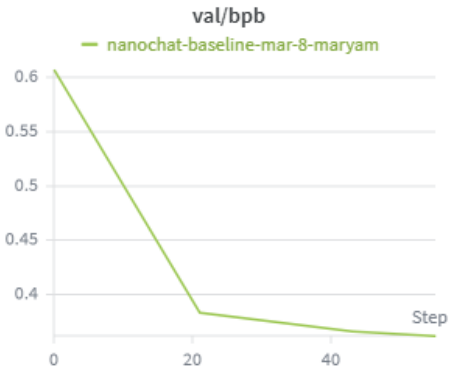

Figure 18:

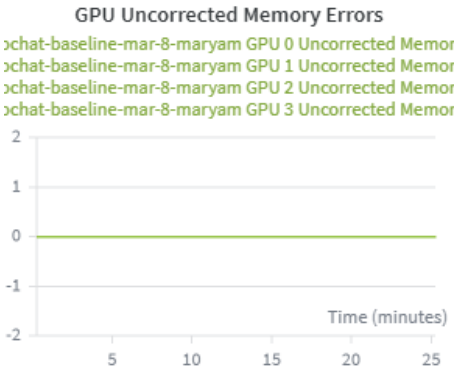

Figure 19:

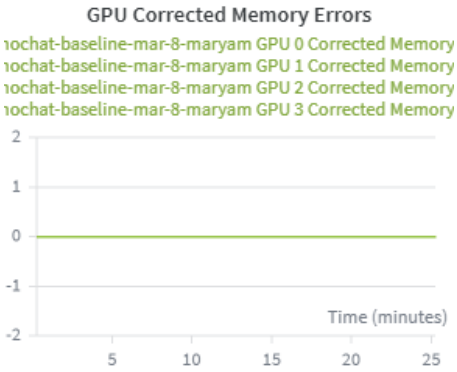

Figure 20:

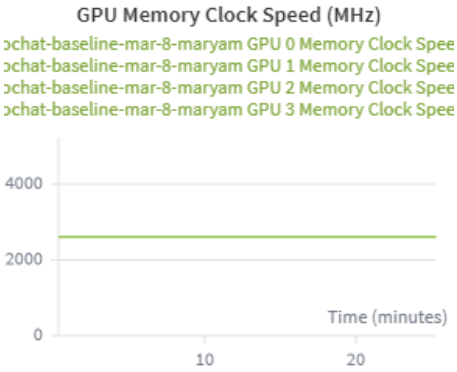

Figure 21:

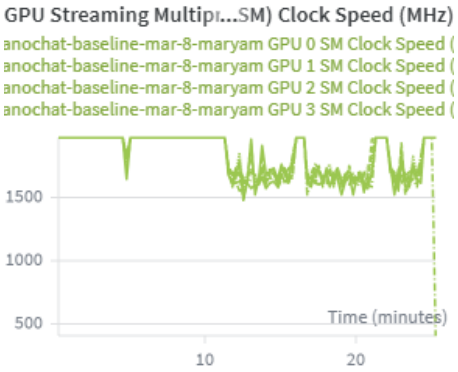

Figure 22:

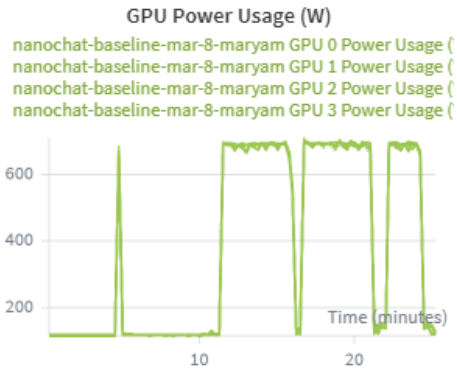

Figure 23:

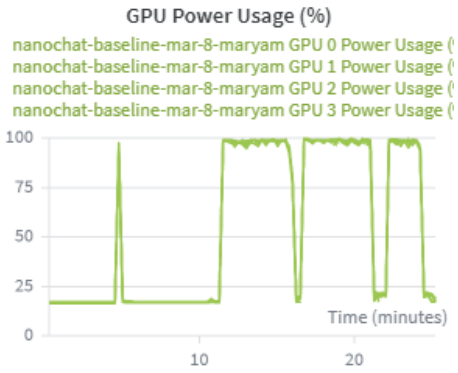

Figure 24:

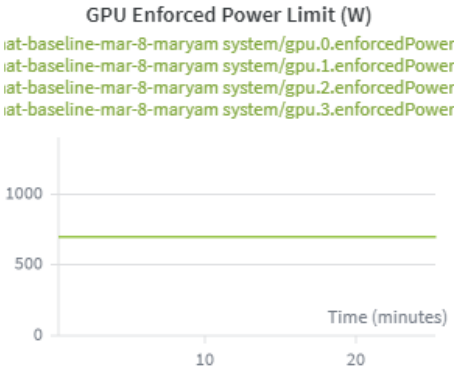

Figure 25:

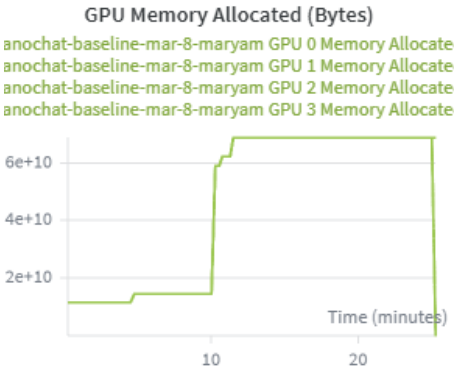

Figure 26:

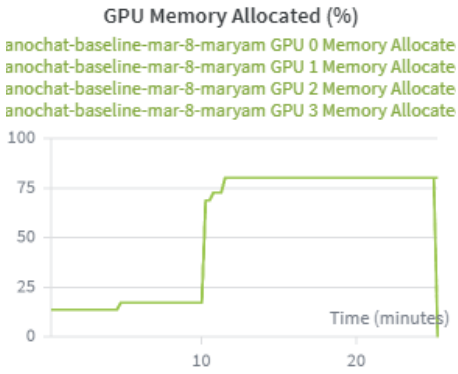

Figure 27:

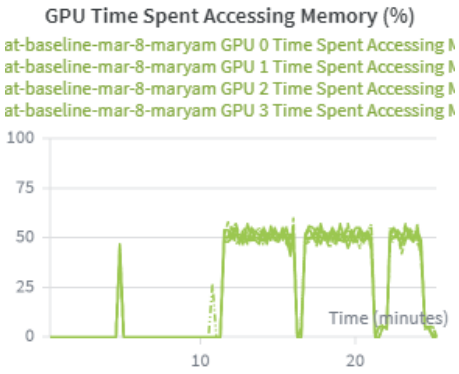

Figure 28:

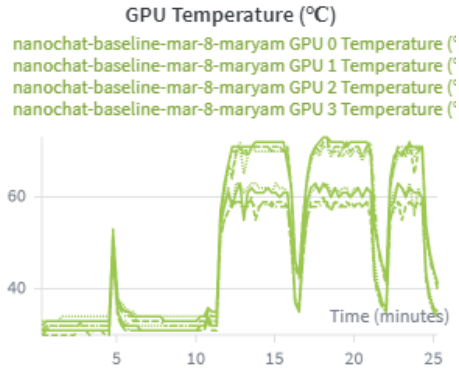

Figure 29:

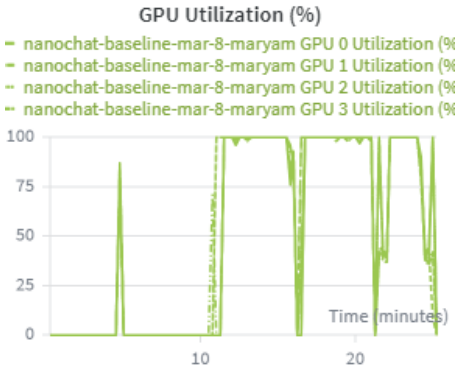

Figure 30:

1 Experiment A: SFT run

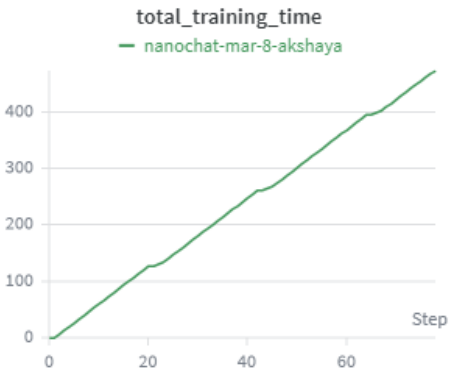

Figure 1:

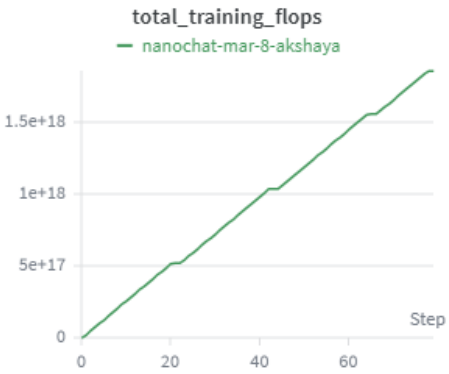

Figure 2:

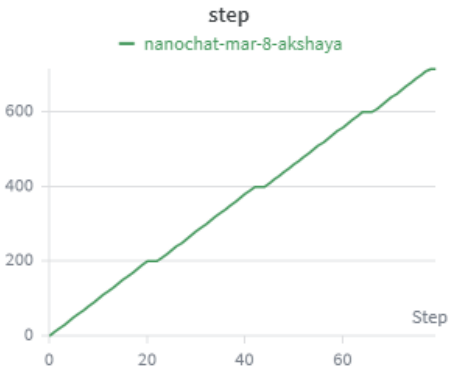

Figure 3:

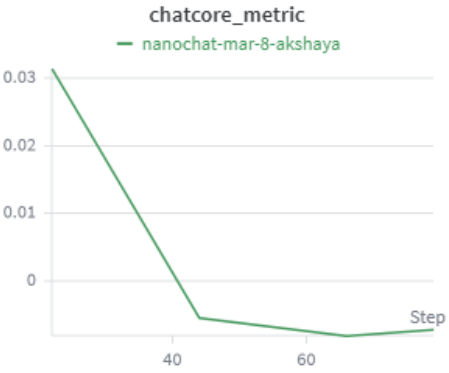

Figure 4:

References

[1] Weights and biases.

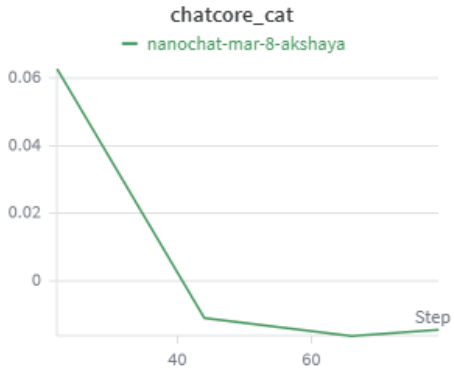

Figure 5:

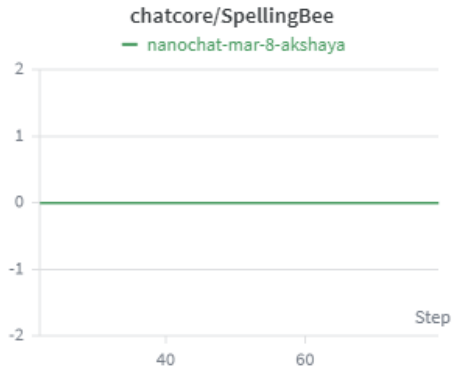

Figure 6:

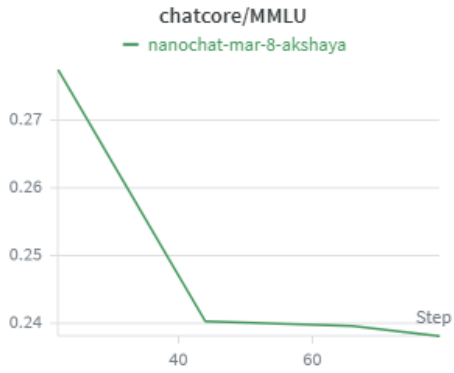

Figure 7:

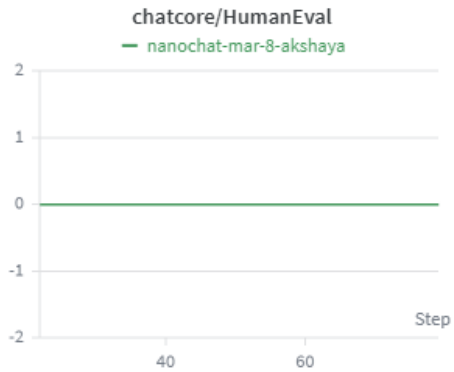

Figure 8:

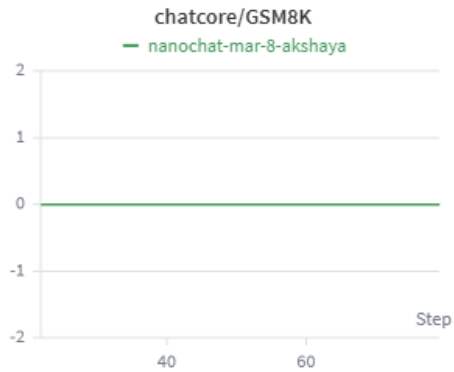

Figure 9:

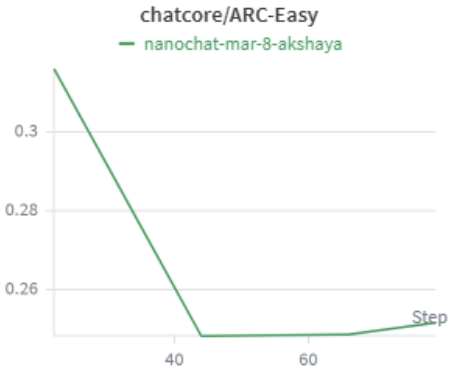

Figure 10:

Figure 11:

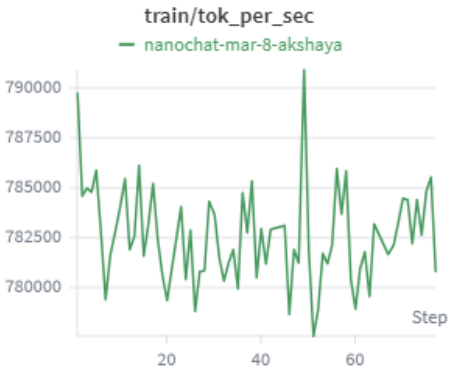

Figure 12:

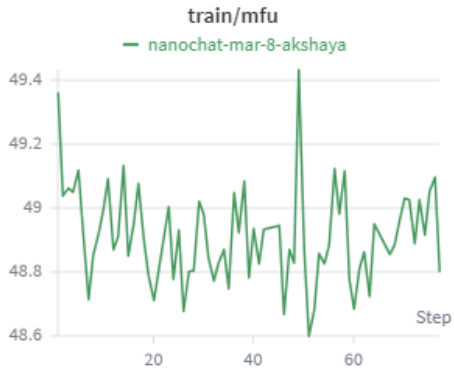

Figure 13:

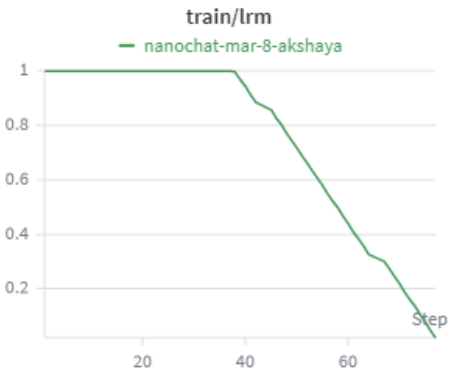

Figure 14:

Figure 15:

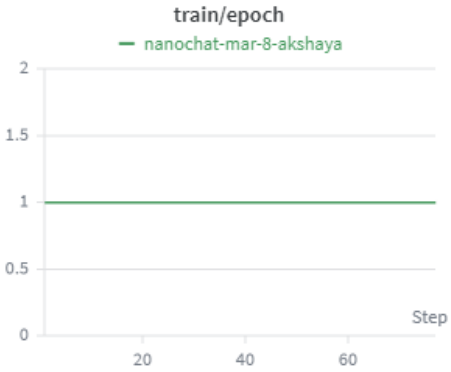

Figure 16:

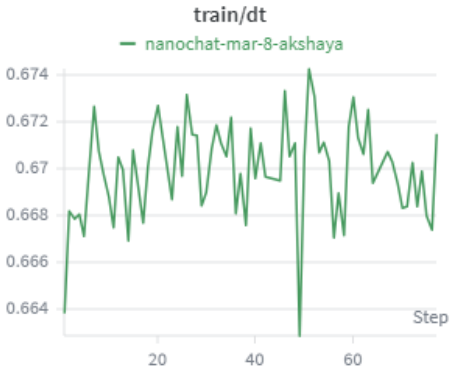

Figure 17:

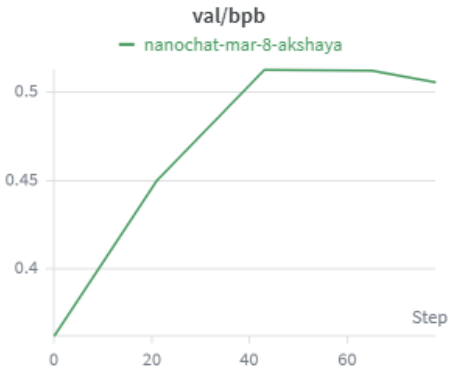

Figure 18:

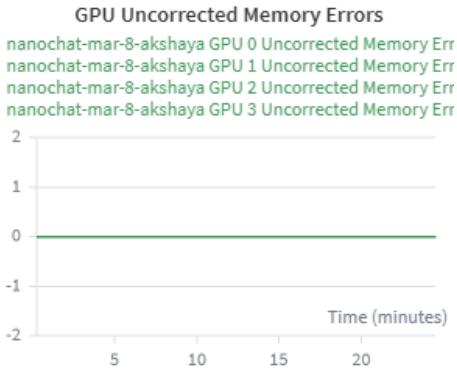

Figure 19:

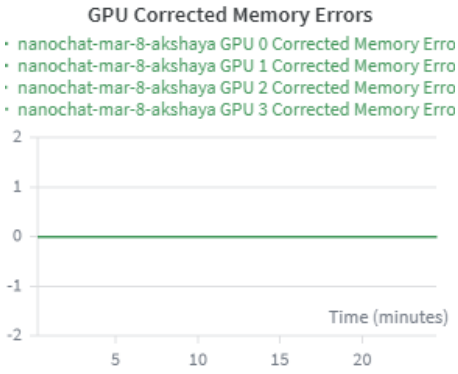

Figure 20:

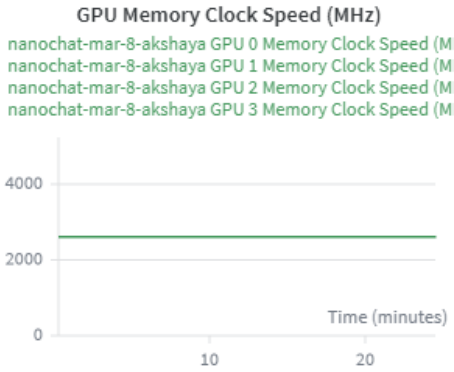

Figure 21:

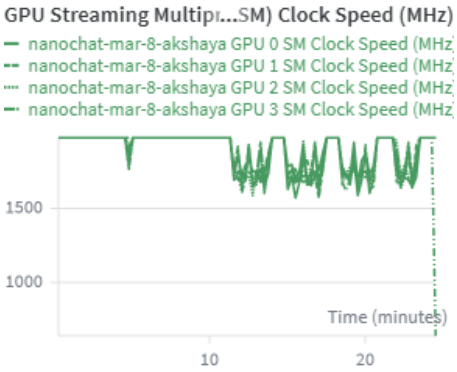

Figure 22:

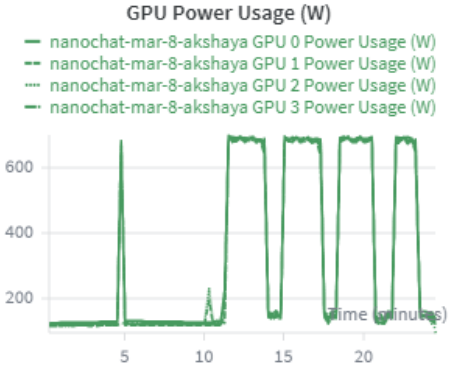

Figure 23:

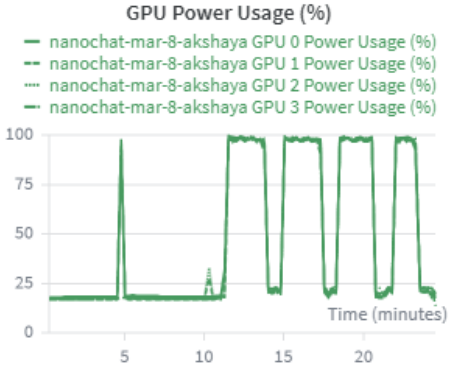

Figure 24:

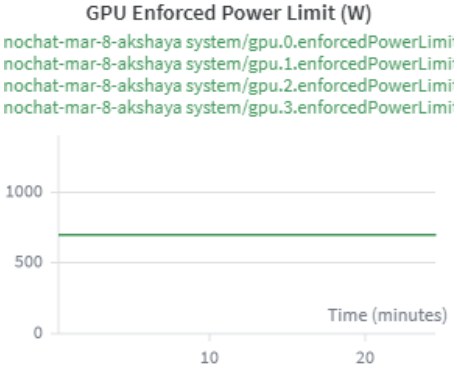

Figure 25:

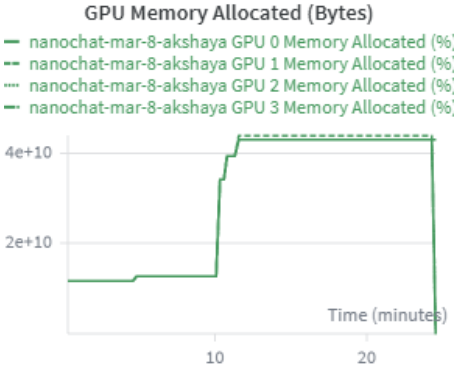

Figure 26:

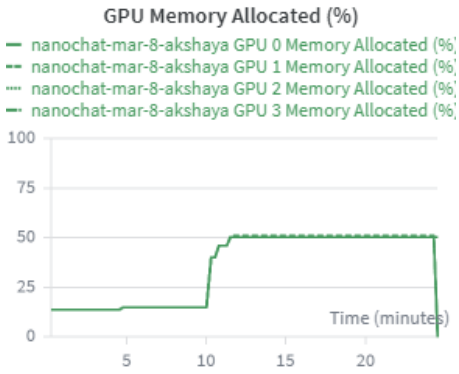

Figure 27:

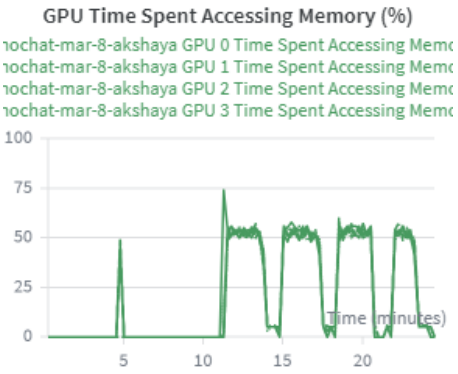

Figure 28:

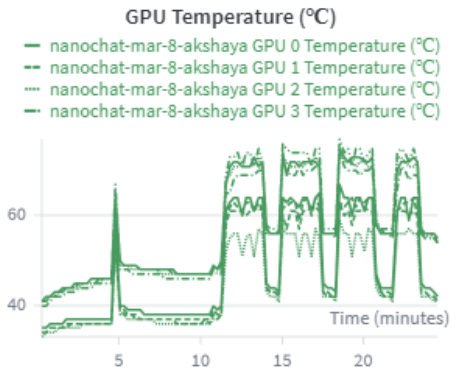

Figure 29:

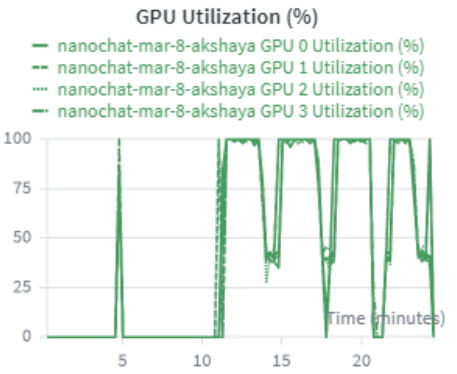

Figure 30:

1 Experiment B SFT run

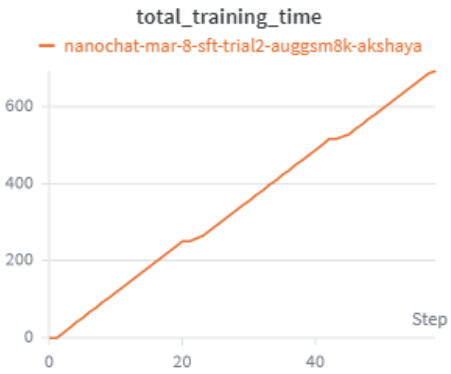

Figure 1:

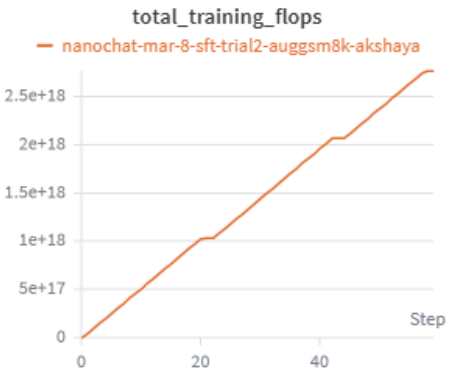

Figure 2:

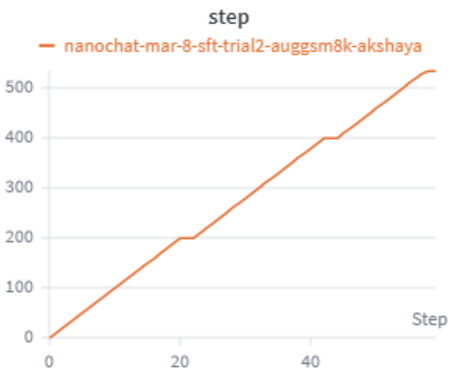

Figure 3:

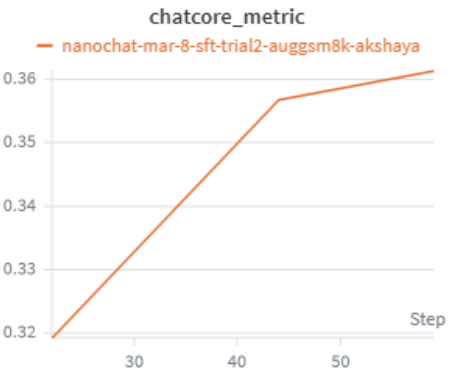

Figure 4:

References

[1] Weights and biases.

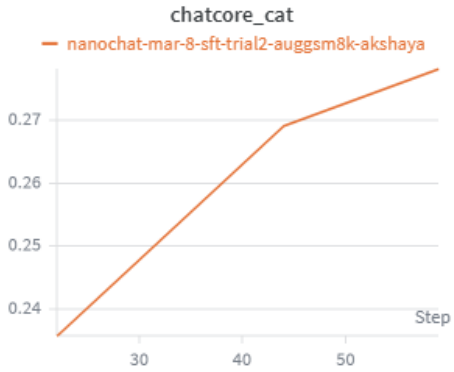

Figure 5:

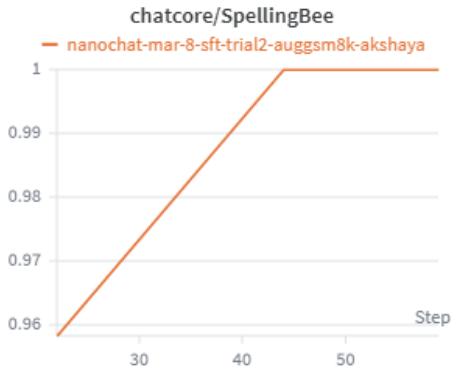

Figure 6:

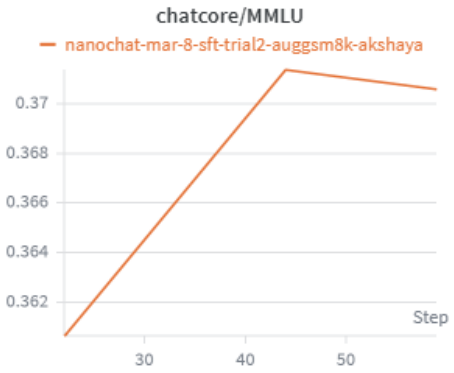

Figure 7:

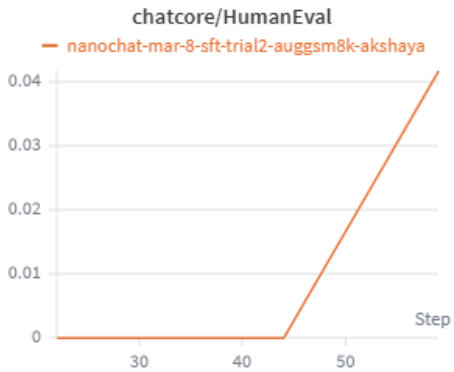

Figure 8:

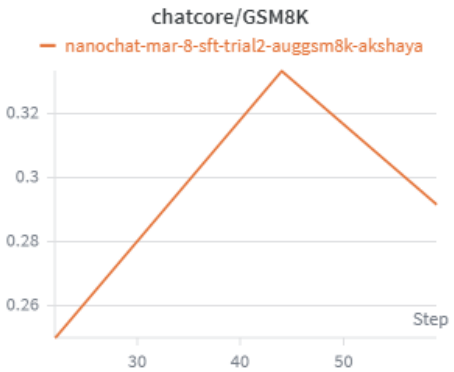

Figure 9:

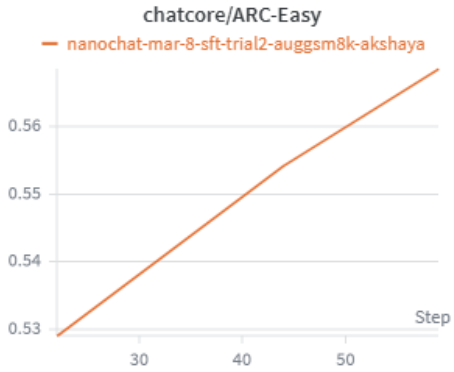

Figure 10:

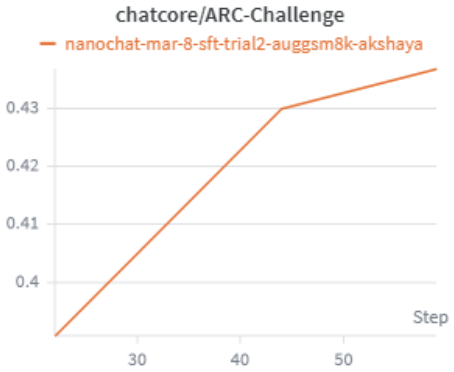

Figure 11:

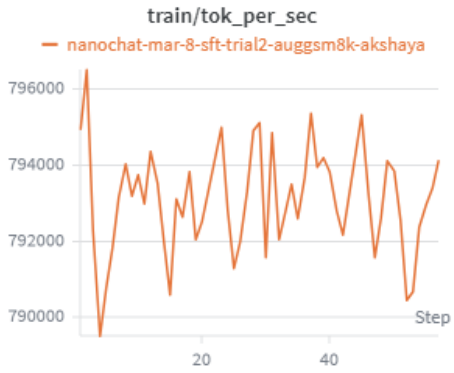

Figure 12:

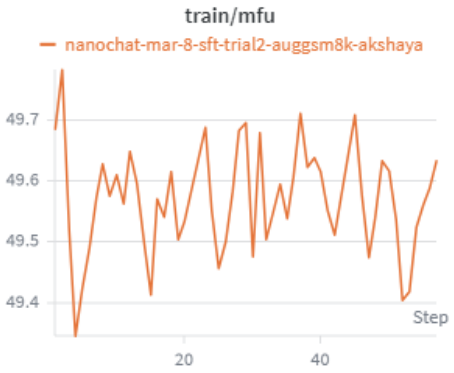

Figure 13:

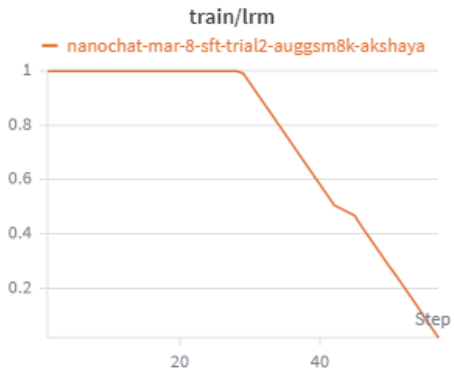

Figure 14:

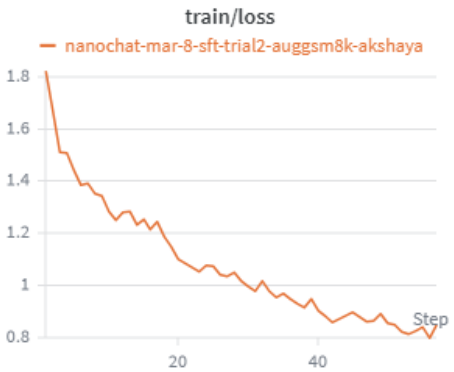

Figure 15:

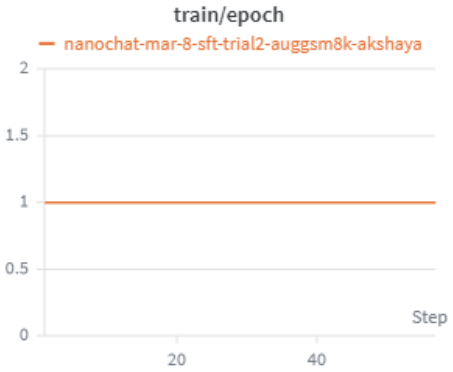

Figure 16:

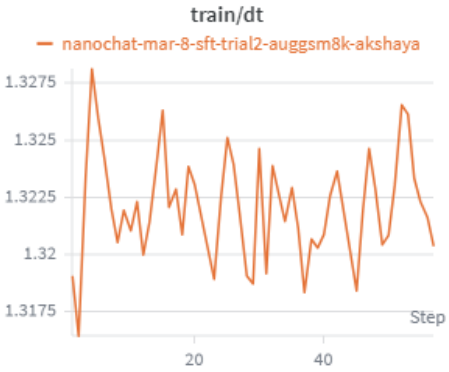

Figure 17:

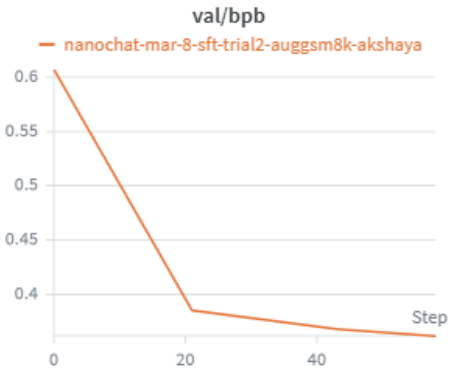

Figure 18:

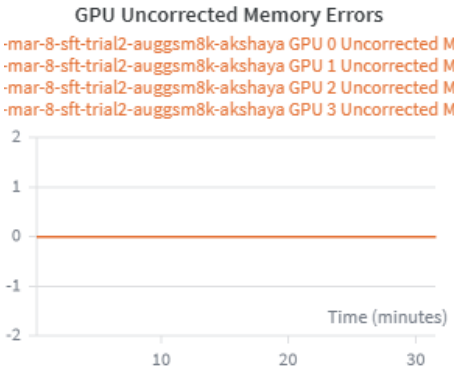

Figure 19:

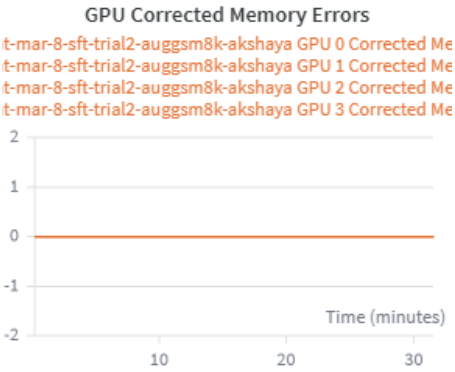

Figure 20:

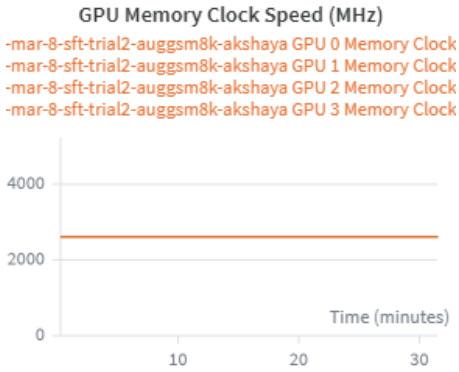

Figure 21:

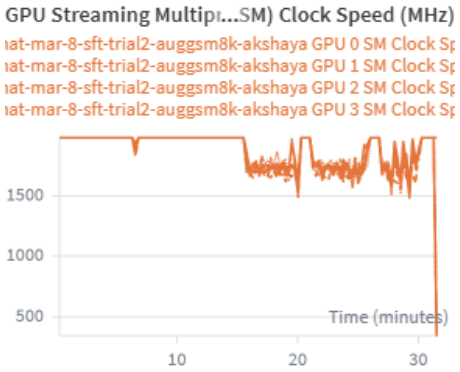

Figure 22:

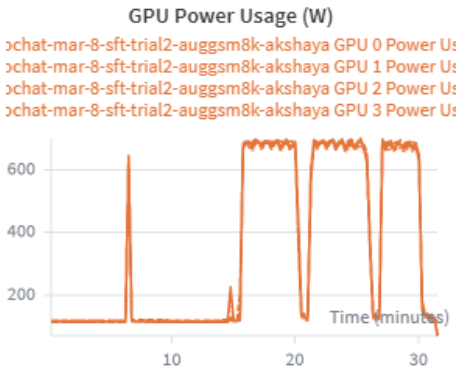

Figure 23:

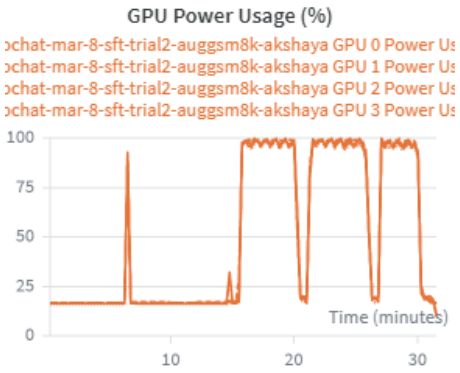

Figure 24:

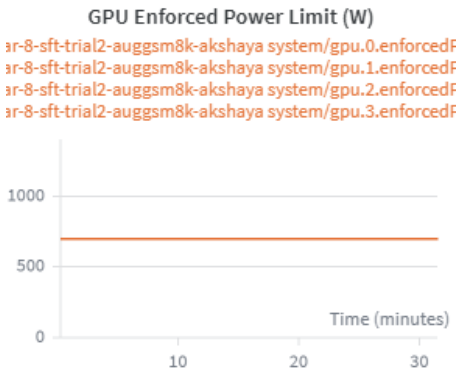

Figure 25:

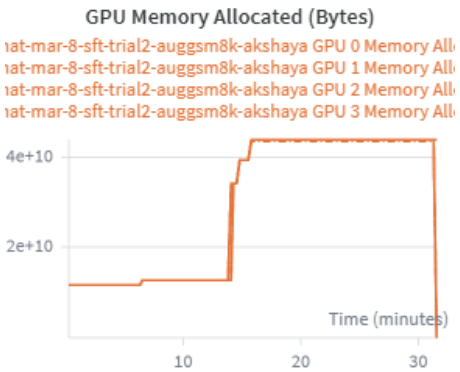

Figure 26:

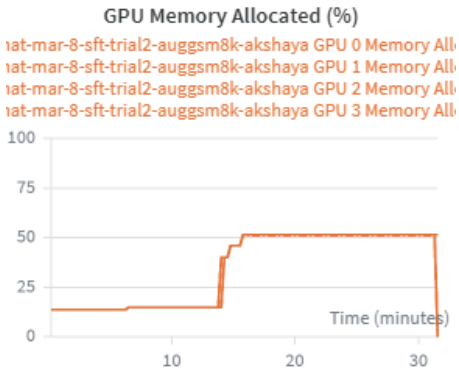

Figure 27:

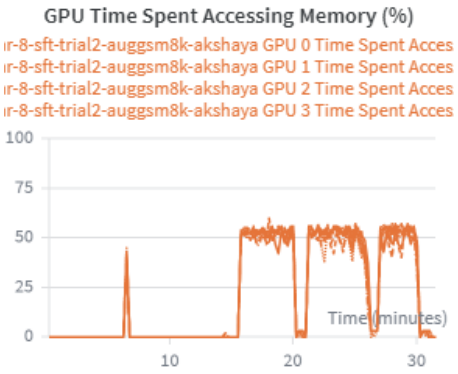

Figure 28:

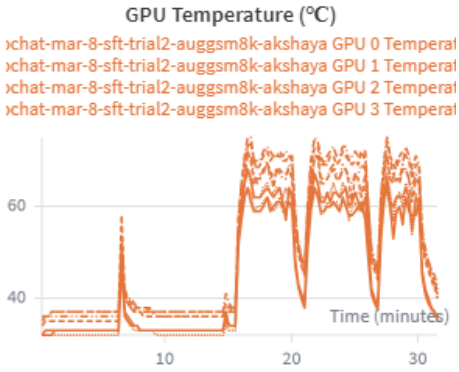

Figure 29:

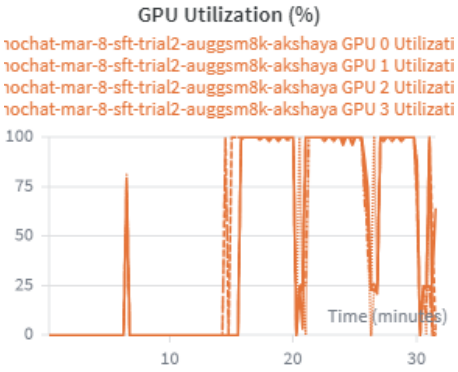

Figure 30:

1 Our Training RL Run Graphs

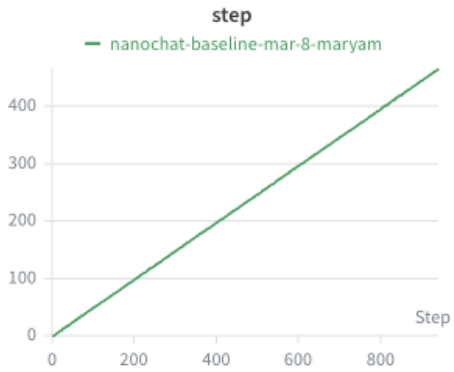

Figure 1:

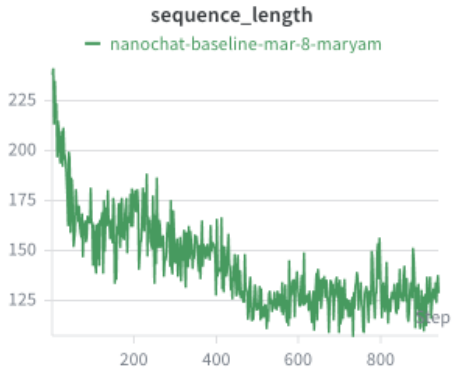

Figure 2:

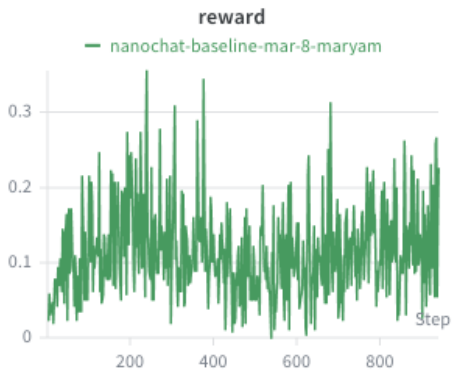

Figure 3:

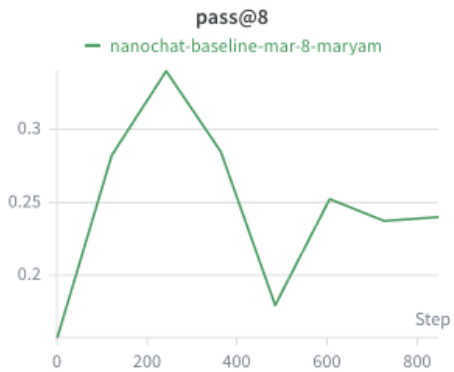

Figure 4:

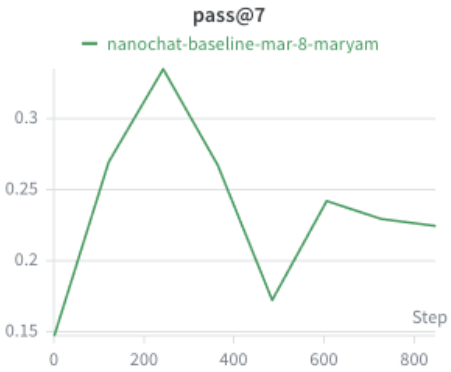

Figure 5:

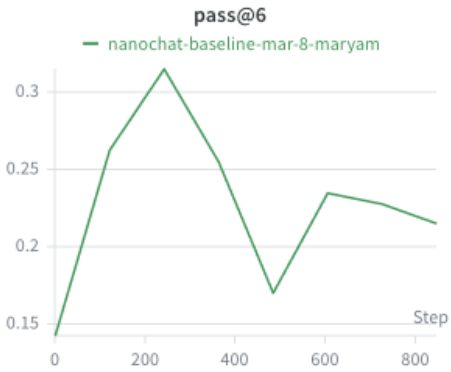

Figure 6:

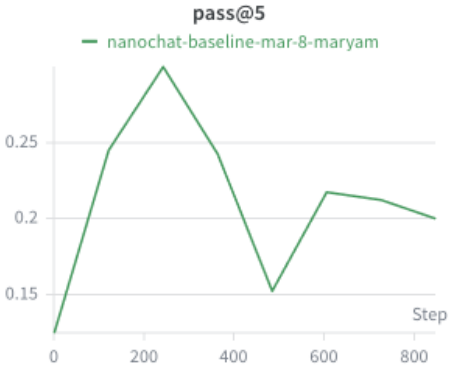

Figure 7:

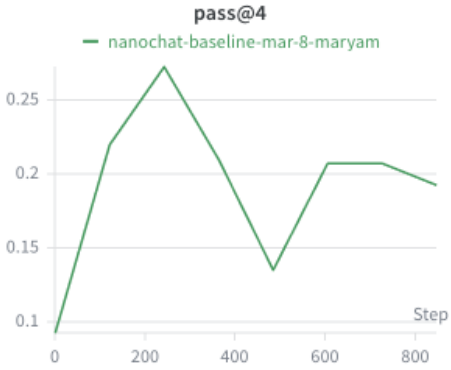

Figure 8:

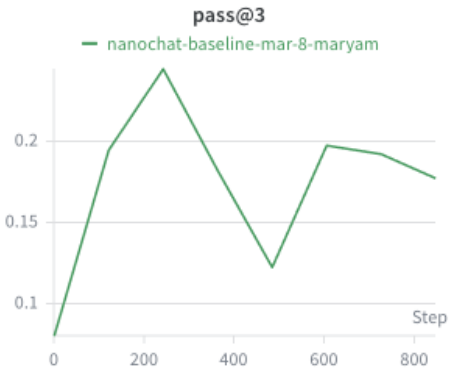

Figure 9:

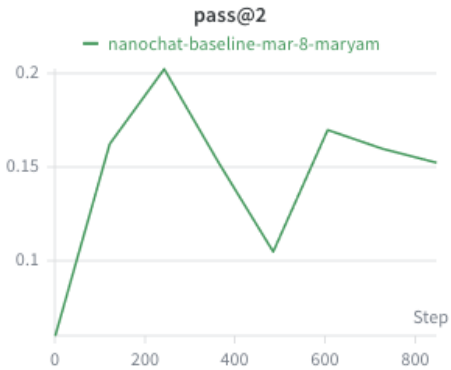

Figure 10:

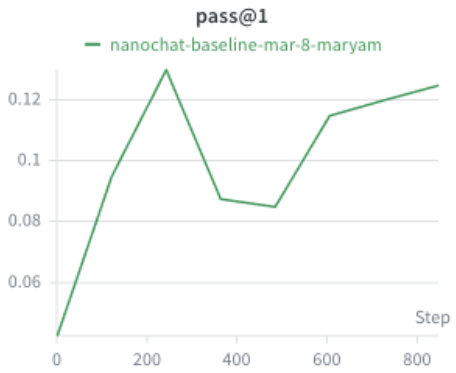

Figure 11:

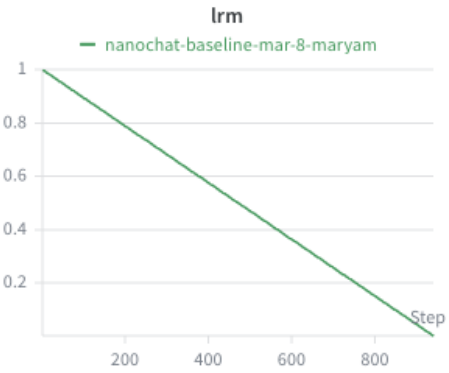

Figure 12:

1 Baseline Nanochat RL

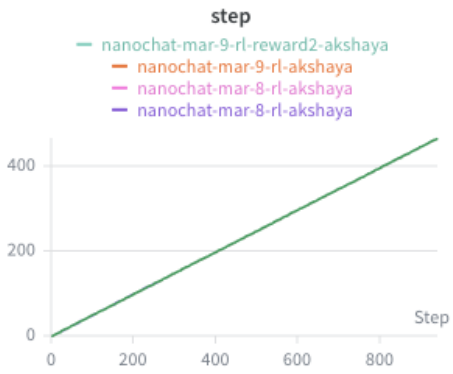

Figure 1:

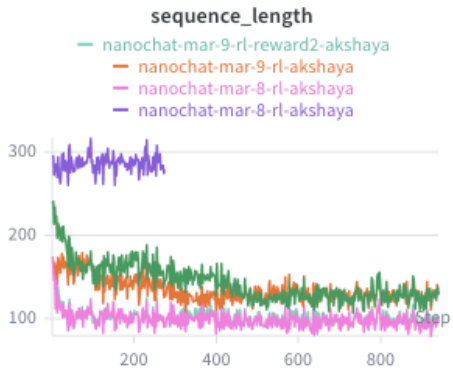

Figure 2:

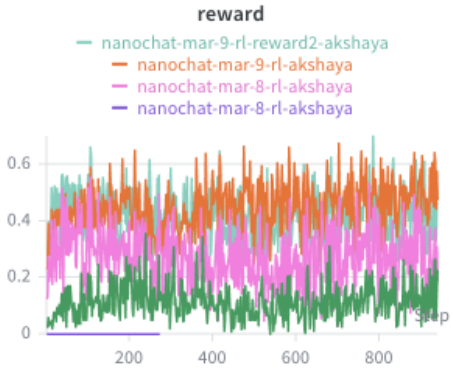

Figure 3:

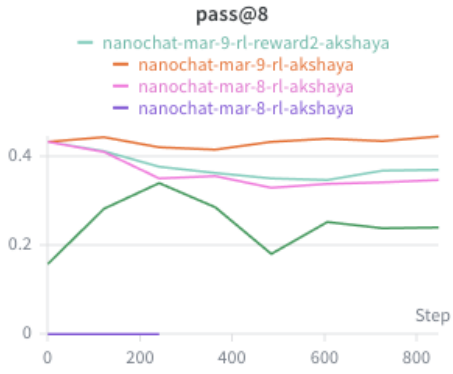

Figure 4:

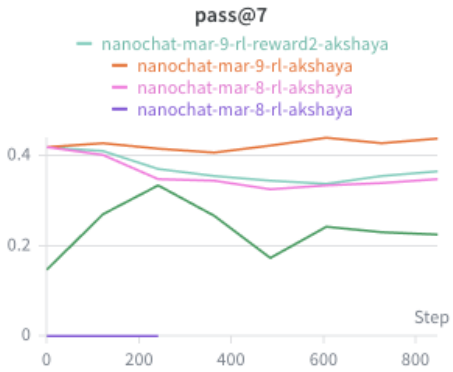

Figure 5:

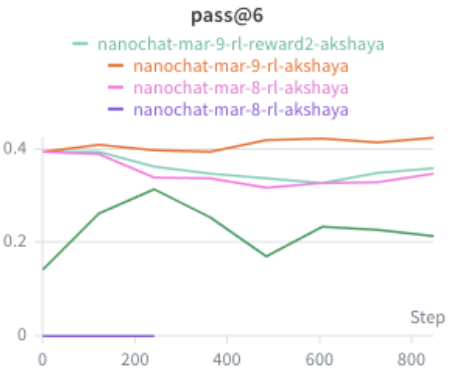

Figure 6:

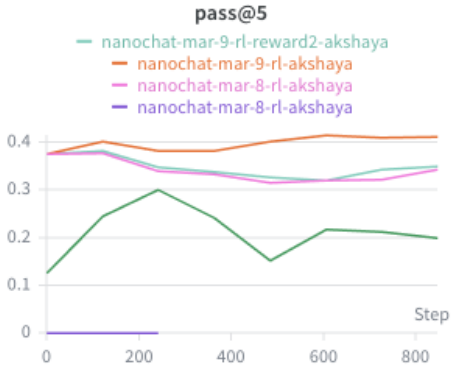

Figure 7:

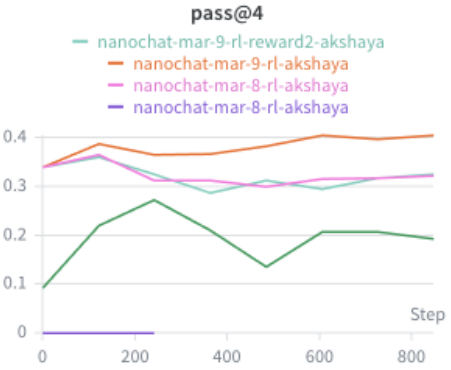

Figure 8:

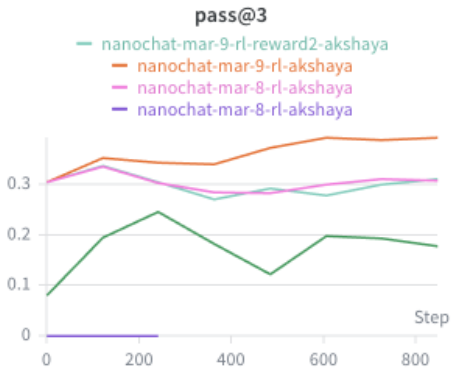

Figure 9:

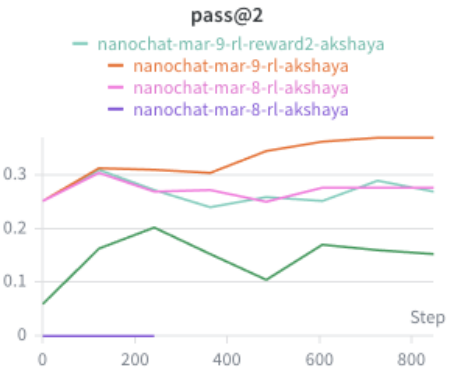

Figure 10:

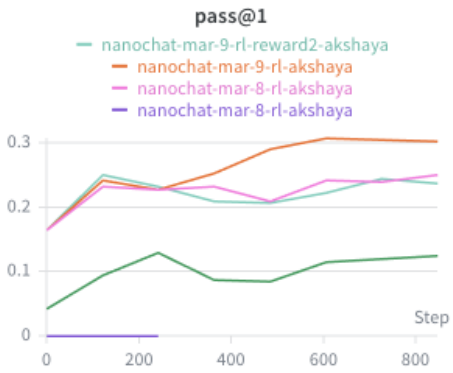

Figure 11:

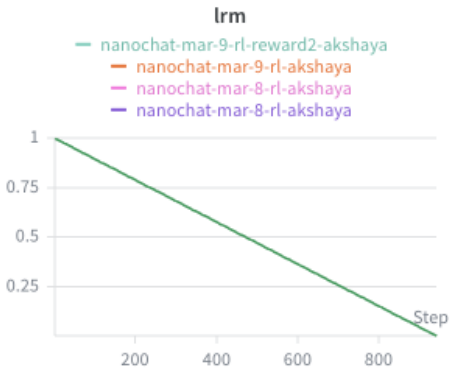

Figure 12:

1 RL - Reward 1

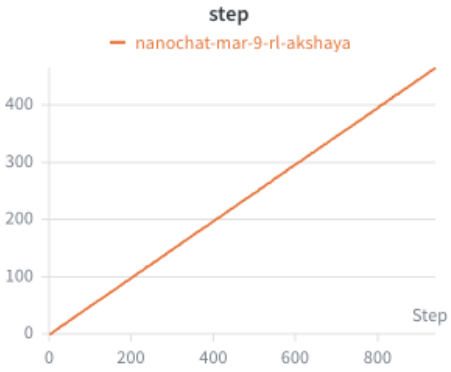

Figure 1:

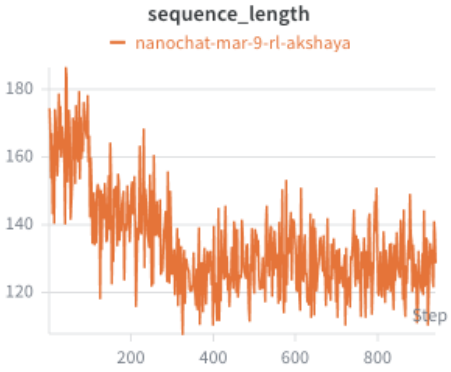

Figure 2:

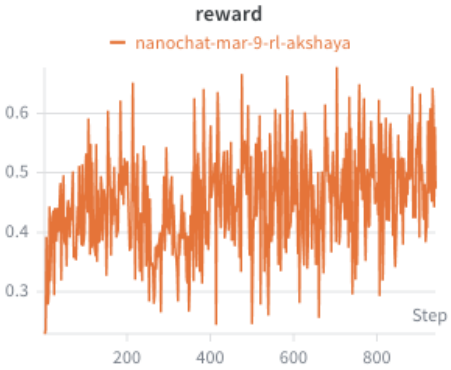

Figure 3:

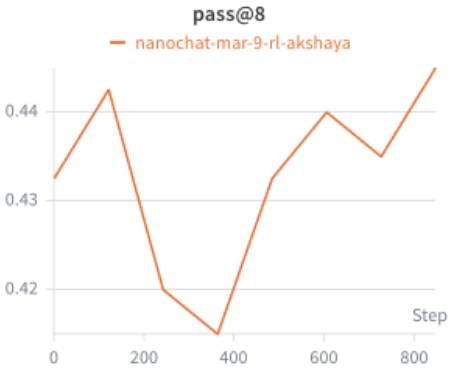

Figure 4:

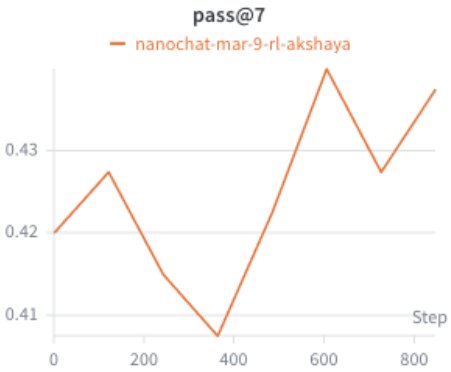

Figure 5:

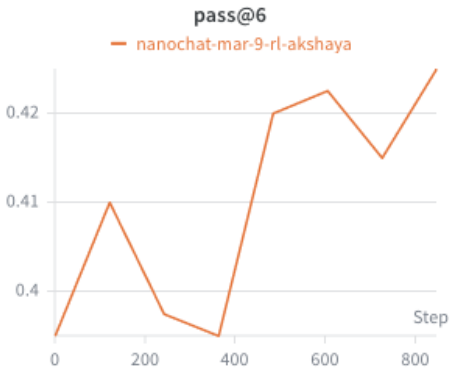

Figure 6:

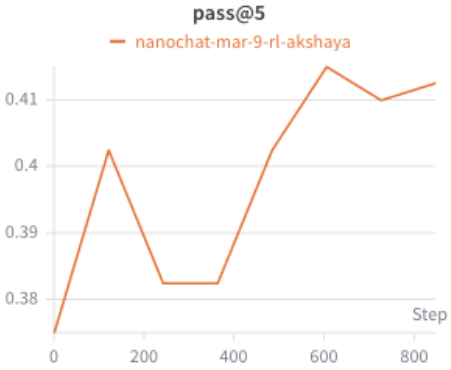

Figure 7:

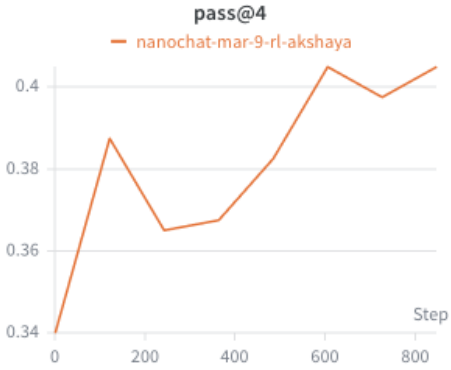

Figure 8:

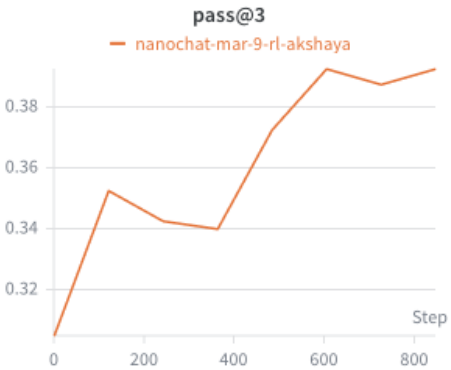

Figure 9:

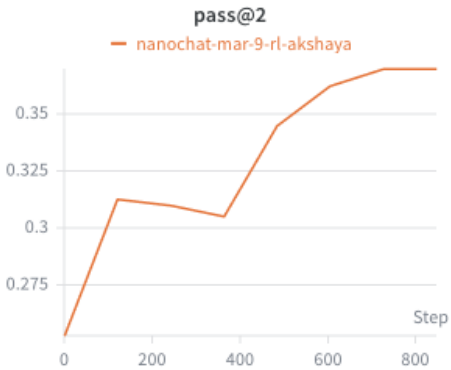

Figure 10:

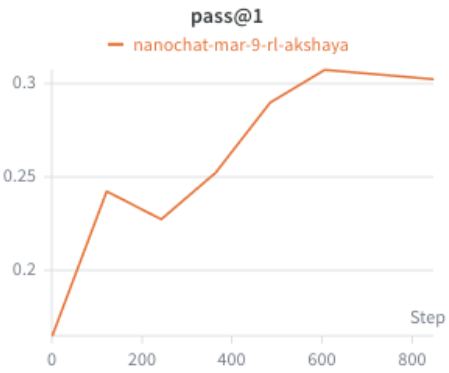

Figure 11:

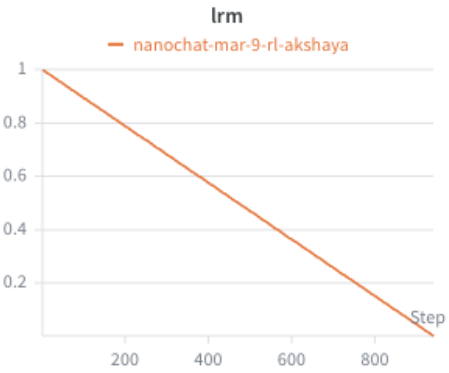

Figure 12:

1 RL - Reward 2

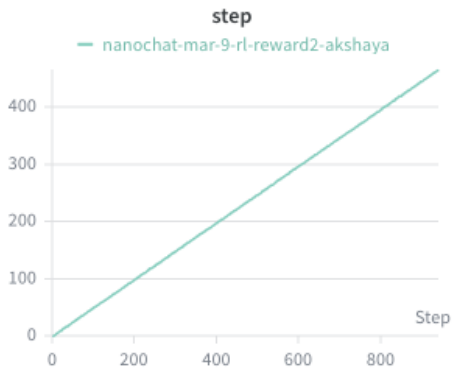

Figure 1:

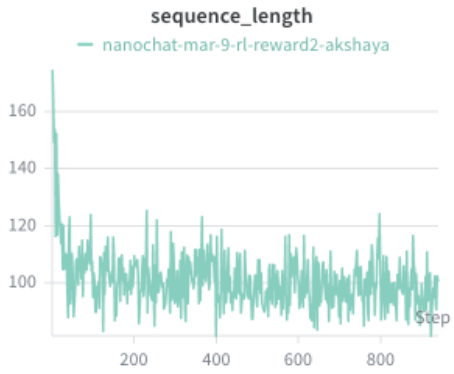

Figure 2:

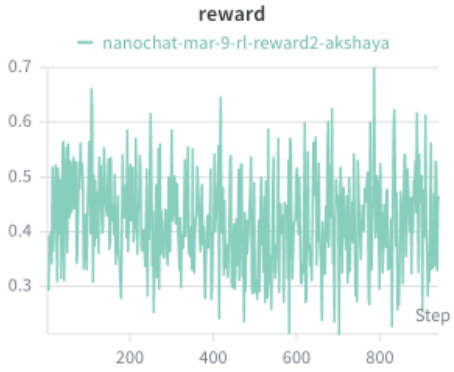

Figure 3:

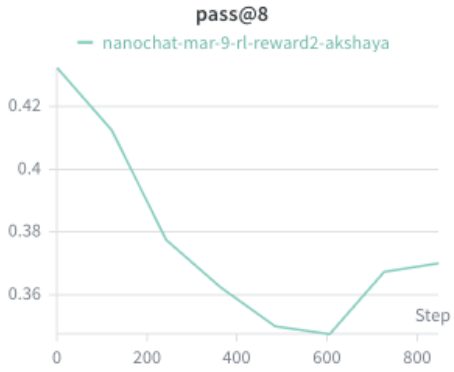

Figure 4:

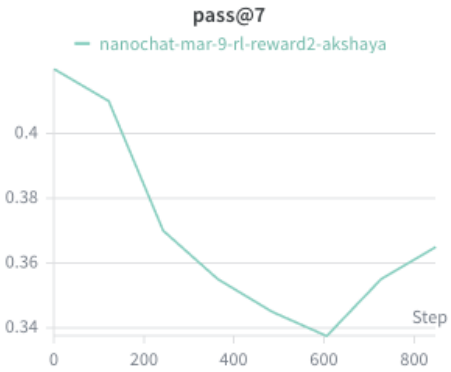

Figure 5:

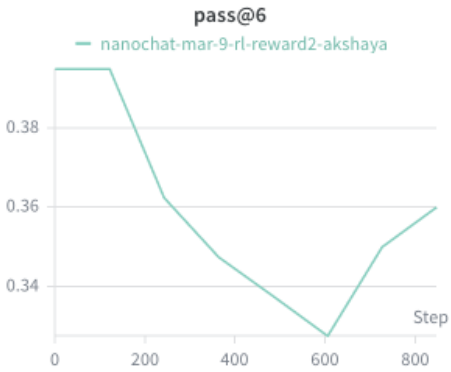

Figure 6:

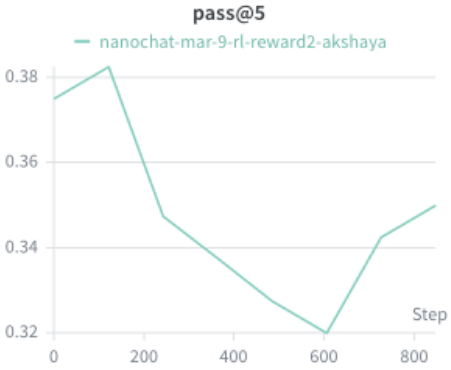

Figure 7:

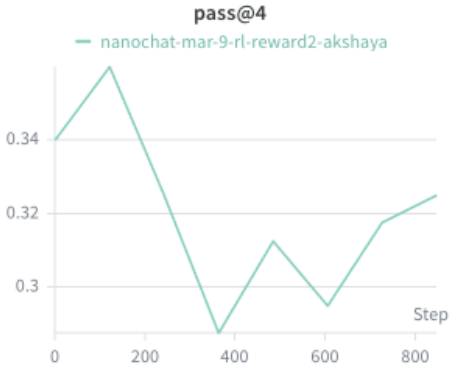

Figure 8:

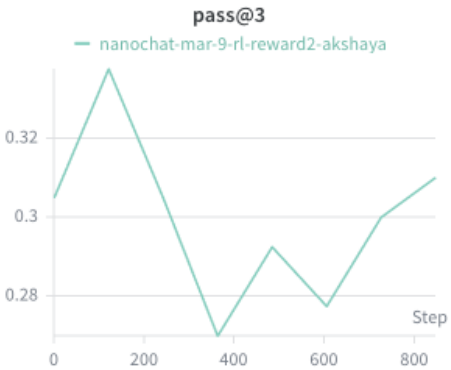

Figure 9:

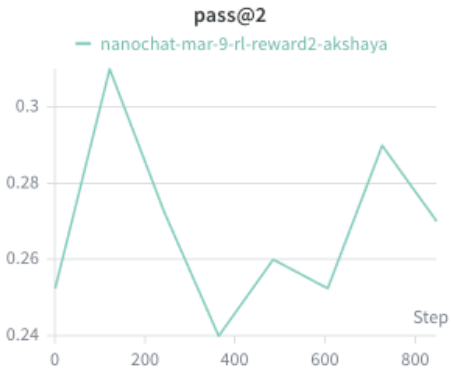

Figure 10:

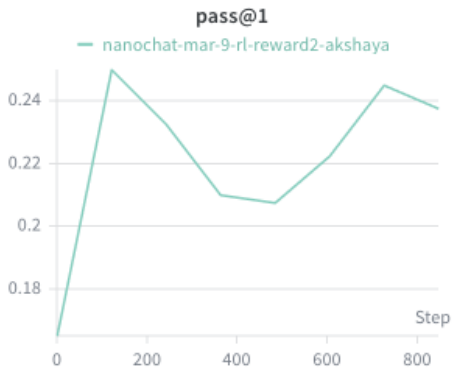

Figure 11:

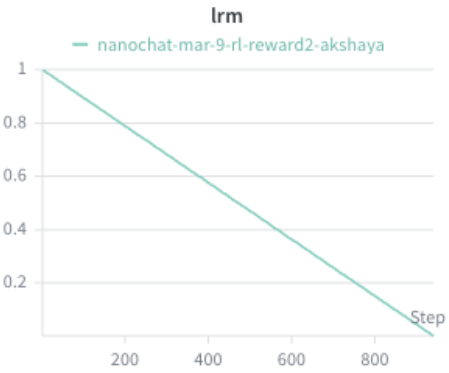

Figure 12:

1 RL - Both Rewards

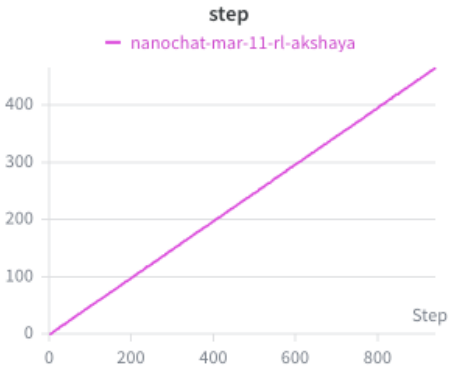

Figure 1:

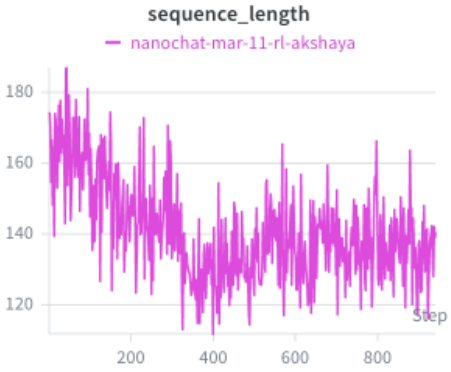

Figure 2:

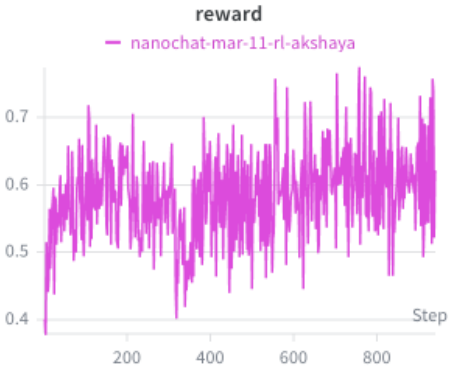

Figure 3:

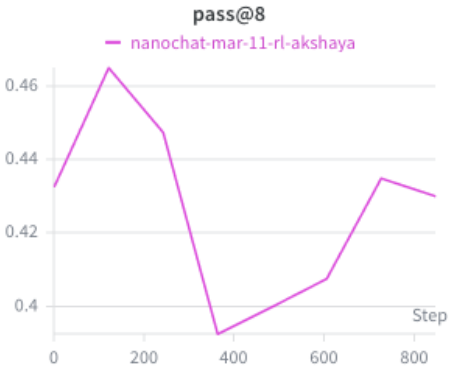

Figure 4:

Figure 5:

Figure 6:

Figure 7:

Figure 8:

Figure 9:

Figure 10:

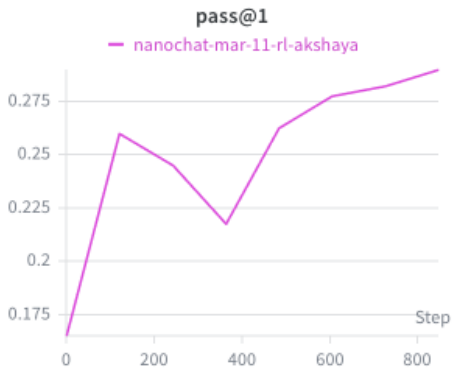

Figure 11:

Figure 12: